

MONITOR Z · CUADERNO COMPLETO

Teoría y láminas del monitor de jobs en z/OS

Lecciones escritas hasta el 2026-10-05: 00, L00, L01, L02, L03, L04, L05, L06, L07, L08, L09, L10, L11

Ruta de aprendizaje

Doce módulos, de "qué es un demonio" a "un STC que avisa al instante y el portal que aprende de él". **Cada módulo termina con algo que corre en DESA**, no sólo con teoría. El orden sigue el plan ya acordado en IDEA-005:

1. Primero **validar sin puerto ni APF** (modo spool).
2. Después agregar el socket.
3. Recién al final convertirlo en STC autorizado con eventos ENF 70.

Así cada paso depende sólo de lo que ya se probó.

Mód.	Tema	Qué aprendés	Qué queda implementado / medido	Requiere de sistemas
0	<i>L00 - Que es un demonio en z-OS y que vamos a construir</i>	Demonio en Unix vs. STC y job largo en z/OS, las piezas del monitor y los 3 niveles (polling, job que duerme, eventos)	El mapa y el diagnóstico de dónde estamos	—
1	<i>L01 - Anatomía de un programa Assembler - el prologo de ZTKMONS</i>	Convención de enlace, save areas, registros base, PARM, AMODE/RMODE, DCB	Leer y explicar el prólogo y el epílogo de ZTKMONS	—
2	<i>El bucle: dormir sin gastar CPU</i>	STIMER, WAIT / POST, el ECB como semáforo (el sem_timedwait de SOPES2 en z/OS)	Un programa mínimo que despierta cada N s, escribe la hora y termina a los M min. Medir su CPU	—
3	<i>Hablar con JES: el SSI</i>	IEFSSREQ, SSOB/SSIB, función 80 (extended status), STATTERS y STATVRBO, las áreas que devuelve y cómo liberarlas	Leer ZTKJACT y la rutina LLAMASSI de ZTKMONS	—

4	ZTKMONS v0 en DESA (modo spool)	Ensamblar y enlazar (ASMA90, binder), leer los errores, correr la demo de vivo\host\LEEME.md	Primera validación real: spool_a_portal.py <JOBID> con ✓/X, y las 5 preguntas abiertas contestadas	—
5	<i>Diseño del protocolo</i>	Por qué texto EBCDIC con X'15', idempotencia, reconexión (ACT, HIST), latido, versionado	Revisar PROTOCOLO.md con lo que enseñó la demo; v1.1 si hace falta	—
6	<i>Sockets en z/OS</i>	API EZASMI/EZASOKET: SOCKET, BIND, LISTEN, ACCEPT, READ y WRITE; bloqueo y plazos (el CONNECT de 77 s de ZTKAVISA)	ZTKMON escuchando en un puerto de DESA y el puente.py real conectado (sin simulador)	Puerto TCP
7	<i>Ser un STC de verdad</i>	PROCLIB, START, comandos del operador (F y P con EXTRACT COMM, CIB y QEDIT), ESTAE/recuperación, reentrancia	ZTKMON arranca con S ZTKMON, atiende F ZTKMON, ESTADO y para limpio con P ZTKMON	STC + PROCLIB
8	<i>Eventos de verdad: ENF 70</i>	APF y estado supervisor, ENFREQ LISTEN, listen exit y POST del ECB, el mapeo IAZENF70, cuidados de un exit (corto, sin I/O)	Aviso instantáneo de INI y FIN sin polling. Comparar latencia y CPU con el modo polling	Librería APF
9	<i>¿Quién tiene el archivo? (ENQ y GQSCAN)</i>	Recursos SYSDSN, ENQ y DEQ, GQSCAN para ver dueños y esperas, sin encolarse nunca	Alerta de contención (IDEA-008): el caso FDCICS02/CICSFILE del 05-10 visto en vivo	— (probar si alcanza estado problema)
10	<i>El lado PC: puente, portal y ML en línea</i>	puente.py, vivo.py, umbral de colgado con Kaplan-Meier, por qué no se aprende de la propia anomalía	Regresión prueba_e2e.py 11/11 contra el STC real	—

11	<i>Operación y pase</i>	Medir CPU por vuelta y por evento, gestión de cambios, runbook, qué pasa si el monitor se cae	Informe de la demo y propuesta para PROD (sigue siendo de sólo lectura)	Gestión de cambios
----	-------------------------	---	---	--------------------

Ruta de aprendizaje: 12 módulos

Monitor z · plan

Mód.	Tema	Qué queda funcionando	Requiere	Estado
0	Qué es un demonio en z/OS	El mapa	—	escrita
1	Anatomía Assembler: prólogo y epílogo	Práctica P01: ZTKHOLA	—	escrita
2	Dormir sin gastar CPU: STIMER, WAIT/POST, ECB	Un bucle con su CPU medida	—	
3	Hablarle a JES: SSI 80	Leer ZTKJACT y LLAMASSI	—	
4	ZTKMONS v0 en DESA (modo spool)	Primera validación real + las 5 preguntas	—	
5	Diseño del protocolo	PROTOCOLO.md revisado	—	
6	Sockets en z/OS (EZASMI)	Monitor conectado al portal real	puerto TCP	
7	Ser un STC: F/P, ESTAE, reentrancia	S / F / P ZTKMON	STC + PROCLIB	
8	Eventos de verdad: ENF 70	Aviso instantáneo; comparar con el polling	APF	
9	¿Quién tiene el archivo? ENQ/GQSCAN	Contención en vivo (IDEA-008, FDCICS02)	—	
10	Lado PC: puente, portal, ML en línea	Regresión 11/11 contra el monitor real	—	
11	Operación y pase	Informe y propuesta (sigue siendo de sólo lectura)	gestión de cambios	

Analogía: subir una montaña por campamentos

Cada módulo es un campamento: no se sube al siguiente sin haber armado la carpa en el anterior. Los campamentos 0–5 se hacen **sin pedir permisos**; del 6 en adelante hace falta que **sistemas** abra la ruta (puerto, STC, APF).

Fuente: Ruta de aprendizaje (cerebro monitor-z)

Lámina R-01_ruta_de_aprendizaje

Estado

- [x] Teoría de los 12 módulos escrita (2026-10-05).
- [] Prácticas: P01 (ZTKHOLA) es la próxima.

Dónde está el código hoy: ZTKMONS v0 (944 líneas) está **escrito y nunca se ensambló**. El puente, el simulador y el portal andan: la regresión dio 11/11 el 2026-09-30. Detalle en [L00 - Que es un demonio en z-OS y que vamos a construir](#).

Criterio propio: el orden $2 \rightarrow 3 \rightarrow 4$ es para que el primer ensamblado (mód. 4) ya lo entiendas línea por línea. Si preferís ensamblar primero y entender después, se puede adelantar el 4.

Glosario

Una línea por término. La lección donde se explica a fondo va entre paréntesis.

Término	Qué es
Address space	El "proceso" de z/OS: un espacio de memoria propio con sus tareas. Cada job y cada STC tiene el suyo (L00, L07)
AMODE / RMODE	Con cuántos bits direcciona el programa (24/31/64) y dónde puede cargarse (bajo o sobre la línea de 16 MB) (L01)
APF	<i>Authorized Program Facility</i> : librerías cuyos programas pueden pedir servicios autorizados (estado supervisor). Lo necesita ENF 70 (L08)
Base + desplazamiento	Forma de direccionar: registro base + 12 bits (0–4095), por eso un registro base cubre 4 KB (L01)
CIB	<i>Command Input Buffer</i> : por donde le llegan a un STC los comandos F (MODIFY) y P (STOP) del operador (L07)
DCB	<i>Data Control Block</i> : describe un archivo para QSAM/BSAM. El de QSAM tiene que estar bajo la línea (L01)
DSECT	Plantilla de un área de memoria, sin reservarla: se "apoya" sobre una dirección con USING (L01, L03)
EBCDIC / X'15'	Código de caracteres de z/OS. X'15' (NL) es el fin de línea del protocolo ZTKMON (L05)
ECB	<i>Event Control Block</i> : una palabra que hace de semáforo. WAIT duerme hasta que alguien hace POST (L02)
ENF 70	Señal de JES2 cuando un job cambia de estado (SELECT, DESELECT, CHANGE, PURGE). Se escucha con ENFREQ LISTEN, que requiere autorización (L08)
ENQ / GQSCAN	ENQ serializa recursos (por ejemplo, SYSDSN + nombre de dataset). GQSCAN lista quién los tiene y quién espera (L09)
ESTAE	Rutina de recuperación: se ejecuta si el programa abenda, para limpiar y avisar (L07)
Iniciador	Address space de JES que ejecuta jobs batch de ciertas clases. Un job largo lo ocupa todo el tiempo (L00)
JES2	El subsistema que recibe, encola, ejecuta y guarda la salida (spool) de los jobs (L03)

Reentrante	Programa que no modifica su propio código ni sus datos en la CSECT. Varias tareas pueden usar la misma copia (L01, L07)
Save area	Bloque de 72 bytes donde se guardan los registros del que llama; se encadenan con HSA y LSA (L01)
SSI / IEFSSREQ	<i>Subsystem Interface</i> : la forma de pedirle cosas a JES desde un programa (L03)
SSI 80	Función 80 del SSI, <i>extended status</i> : estado de jobs. Variantes STATTERS (resumen) y STATVRBO (detalle). No requiere APF (L03)
SSOB / SSIB	Bloques de control del pedido SSI: qué función se pide (SSOB) y a qué subsistema (SSIB) (L03)
STC	<i>Started task</i> : programa que se arranca con <code>S nombre</code> , vive en su address space y no usa un iniciador batch. Es el "demonio" de z/OS (L00, L07)
STIMER	Macro para esperar un intervalo (<code>STIMER WAIT</code>) o programar una salida por tiempo (L02)
USING	Promesa al ensamblador sobre qué dirección va a tener un registro base (L01)
WTO	<i>Write To Operator</i> : mensaje a la consola y al SYSLOG (L01, L07)

Término	Qué es	Lección
Address space	El "proceso" de z/OS: memoria propia con sus tareas. Cada job y cada STC tiene el suyo	L00, L07
AMODE / RMODE	Con cuántos bits direcciona (24/31/64) y dónde puede cargarse (bajo o sobre los 16 MB)	L01
APF	Librerías autorizadas: sus programas pueden pedir servicios de estado supervisor. Lo necesita ENF 70	L08
Base + desplaz.	Registro base + 12 bits (0–4095): un registro base cubre 4 KB	L01
CIB	Por donde le llegan a un STC los comandos F (MODIFY) y P (STOP)	L07
DCB	Describe un archivo para QSAM/BSAM; el de QSAM va bajo la línea	L01
DSECT	Plantilla de un área, sin reservarla; se apoya con USING	L01, L03
EBCDIC · X'15'	Código de caracteres de z/OS · fin de línea (NL) del protocolo	L05
ECB	Palabra que hace de semáforo: WAIT duerme hasta que otro hace POST	L02
ENF 70	Aviso de JES2 cuando un job cambia de estado; se escucha con ENFREQ LISTEN (autorizado)	L08
ENQ · GQSCAN	Serializar un recurso (SYSDSN + dataset) · listar quién lo tiene y quién espera	L09
ESTAE	Rutina de recuperación si el programa abanda	L07
Iniciador	Address space de JES que ejecuta jobs batch de ciertas clases	L00
JES2	Recibe, encola, ejecuta y guarda la salida (spool) de los jobs	L03
Reentrante	No modifica su código ni sus datos en la CSECT; varias tareas comparten la copia	L01, L07
Save area	72 bytes con los registros del que llama; se encadenan con HSA y LSA	L01
SSI · IEFSSREQ	Subsystem Interface: la forma de pedirle cosas a JES	L03
SSI 80	Extended status de jobs (STATTERS resumen, STATVRBO detalle); sin APF	L03
SSOB · SSIB	Bloques del pedido SSI: qué función (SSOB) y a qué subsistema (SSIB)	L03
STC	Started task: se arranca con S, address space propio, sin iniciador batch	L00, L07
STIMER	Esperar un intervalo (STIMER WAIT) o programar una salida por tiempo	L02
USING	Promesa al ensamblador del valor de un registro base	L01
WTO	Write To Operator: mensaje a la consola y al SYSLOG	L01, L07

Fuente: Glosario (cerebro monitor-z) · manuales de IBM citados en cada lección

Lámina G-01_glosario

Relacionado: [00 - Monitor z \(MOC\)](#) · [L00 - Que es un demonio en z-OS y que vamos a construir](#) · [L01 - Anatomía de un programa Assembler - el prologo de ZTKMONS](#).

 Fuente: definiciones resumidas de los manuales de IBM citados en cada lección (Assembler Services Guide, Using the Subsystem Interface, Authorized Assembler Services Reference, HLASM Language Reference).

L00 — Qué es un demonio en z/OS y qué vamos a construir

Objetivo: al terminar sabés explicar con tus palabras qué es un demonio, cómo se traduce cada idea de Unix al mundo z/OS, qué piezas tiene nuestro monitor y en qué punto está cada una. **Esta lección no tiene código.** Es el mapa para no perderse en las siguientes.

1. El demonio en Unix (la idea que ya conocés)

Un **demonio** (*daemon*) es un proceso que:

1. arranca solo, sin una terminal atada (con el sistema o con un comando);
2. vive mucho tiempo, en general mientras el sistema esté arriba;
3. pasa casi todo el tiempo **esperando** algo: un evento, un timer o una conexión;
4. cuando despierta hace un trabajo corto y vuelve a esperar;
5. obedece señales para que lo controlen desde afuera (`SIGTERM` para parar, `SIGHUP` para releer la configuración);
6. deja rastro en un log (`syslog`).

La clave es el punto 3: **un buen demonio no gasta CPU mientras espera.** No hace `while(1){ preguntar(); }`. Se duerme en `sleep`, `select/poll` o un semáforo, y el sistema operativo lo despierta.

🕒 Analogía: el sereno de un edificio

Llega solo a su turno (nadie lo trae de la mano), se queda toda la noche, pasa casi todo el tiempo **sentado esperando**, cuando suena el timbre hace algo corto y vuelve a sentarse, obedece al administrador ("andate", "cambiá la ronda") y anota todo en su cuaderno.

#	Un demonio...	El sereno...	Nuestro monitor ZTKMON...
1	arranca solo, sin terminal atada	llega solo a su turno	lo arranca el operador con <code>S ZTKMON</code> (versión STC)
2	vive mucho tiempo	se queda toda la noche	vive todo el cierre
3	pasa casi todo el tiempo esperando	está sentado, no camina sin parar	duerme con <code>STIMER</code> o <code>WAIT</code> : CPU ≈ 0
4	despierta, hace algo corto y vuelve a esperar	abre la puerta y se vuelve a sentar	pregunta a JES, manda los cambios y vuelve a dormir
5	obedece órdenes de afuera	"andate" / "cambiá la ronda"	<code>P ZTKMON</code> (parar) y <code>F ZTKMON,...</code> (ordenar)
6	deja rastro en un log	anota en su cuaderno	<code>SYSPRINT</code> y <code>WTO</code> a la consola

La más importante es la 3. Un buen demonio **no gasta CPU mientras espera**: el sistema operativo lo despierta.

Fuente: Lección L00, sección 1

Lámina L00-01_que_es_un_demonio

El bucle: mal demonio vs. buen demonio

❌ Mal demonio: pregunta sin parar

```
mientras siempre:
  preguntar si pasó algo
  (y vuelve a preguntar de inmediato)
```

- Usa la CPU al 100 % aunque no pase nada.
- Le roba tiempo a los jobs del cierre.
- Si encima le pregunta a JES, también lo carga a él.

✅ Buen demonio: duerme y despierta

```
mientras siempre:
  dormir hasta que me despierten
  o pasen N segundos
  ver qué cambió y avisar
```

- Dormido: **CPU ≈ 0**.
- Trabaja sólo un instante en cada vuelta.
- Avisa sólo los **cambios**, no todo de nuevo.

👶 Analogía: el niño en el auto vs. la alarma del celular

El mal demonio es el niño que pregunta "¿ya llegamos?" cada segundo: no avanza más rápido y cansa a todos. El buen demonio pone una **alarma** ("despertame en 5 minutos o cuando lleguemos") y se duerme.

Para dormir, en...	Unix	z/OS	Lo vemos en
Un tiempo fijo	<code>sleep(5)</code>	<code>STIMER WAIT,BINTVL=...</code>	L02
Hasta que me avisen	<code>sem_wait()</code>	<code>WAIT</code> sobre un <code>ECB</code> (otro hace <code>POST</code>)	L02
Lo primero que pase de varias cosas	<code>select()</code> / <code>poll()</code>	<code>WAIT ECBLIST=...</code>	L02, L08

Fuente: Lección L00, sección 1 · z/OS MVS Programming: Assembler Services Guide (STIMER, WAIT/POST)

Lámina L00-02_bucle_malo_vs_bueno

2. La misma idea en z/OS

Idea Unix	Equivalente z/OS	Lo vemos en
Proceso	Address space (espacio de direcciones), con sus tareas (TCB)	L07
Demonio que arranca con el sistema	STC (started task) : el operador o el arranque lo levanta con <code>S ZTKMON</code> . Su JCL sale de una PROCLIB y no ocupa un iniciador batch	L07
Proceso largo lanzado a mano	Job batch largo : ocupa un iniciador de su clase todo el tiempo que corre. Es lo que hace hoy ZTKMONS (v0)	L04
<code>sleep(5)</code>	<code>STIMER WAIT, BINTVL=...</code>	L02
Semáforo (<code>sem_wait / sem_post</code>)	ECB (<i>event control block</i> , una palabra de memoria) con <code>WAIT</code> y <code>POST</code>	L02
<code>select()</code> sobre varios descriptors	<code>WAIT ECBLIST=...</code> : despertar con el primer ECB que se postee	L02, L08
Señales (<code>SIGTERM</code> , <code>SIGHUP</code>)	Comandos del operador <code>P ZTKMON</code> (STOP) y <code>F ZTKMON, texto</code> (MODIFY), que llegan por el CIB (<code>EXTRACT ... FIELDS=COMM, QEDIT</code>)	L07
<code>syslog</code>	<code>WTO</code> a la consola / <code>SYSLOG</code> , y <code>SYSPRINT</code>	L01, L04
Preguntar al kernel por los procesos (<code>ps</code>)	Preguntar a JES2 por los jobs: SSI función 80 (<i>extended status</i>)	L03
Que el kernel avise (<code>inotify</code> , eventos)	ENF 70 : JES2 avisa cuando un job cambia de estado (SELECT, DESELECT, CHANGE, PURGE). Hace falta APF	L08
Ver quién tiene un archivo bloqueado (<code>lsof</code> , <code>flock</code>)	ENQ sobre el recurso <code>SYSDSN</code> y GQSCAN para listar dueños y esperas	L09
Correr como root	APF + estado supervisor / clave 0-7. Sólo lo piden ENF 70 y otros servicios autorizados	L08

 Fuente: los términos y servicios son los de *z/OS MVS Programming: Assembler Services Guide* (STIMER, WAIT/POST, ECB, EXTRACT, QEDIT, ENQ/GQSCAN), *z/OS MVS Using the*

Subsystem Interface (SSI 80) y z/OS MVS Programming: Authorized Assembler Services Reference (ENFREQ). La tabla de equivalencias es criterio propio, para orientarse.

De Unix a z/OS: tabla de equivalencias

L00 · Qué es un demonio en z/OS

Lo que conocés de Unix	En z/OS se llama	En nuestro monitor	Lección
Proceso	Address space (con sus tareas, TCB)	ZTKMON vive en el suyo	L07
Demonio que arranca con el sistema	STC (<i>started task</i>): S ZTKMON, JCL en una PROCLIB, sin iniciador batch	La versión final	L07
Programa largo lanzado a mano	Job batch largo : ocupa un iniciador todo el tiempo	ZTKMONS v0, hoy	L04
<code>sleep(5)</code>	STIMER WAIT	El "dormir" del bucle	L02
Semáforo	ECB + WAIT / POST	Despertar el bucle con eventos	L02
<code>select()</code>	WAIT ECBLIST=...	Timer + evento + orden del operador	L02, L08
Señales (kill , SIGTERM, SIGHUP)	P ZTKMON (STOP) y F ZTKMON,... (MODIFY), por el CIB	Parar u ordenar sin cancelar	L07
<code>syslog</code>	WTO (consola / SYSLOG) y SYSPRINT	El rastro del monitor	L01, L04
<code>ps</code> (listar procesos)	Preguntarle a JES2 por SSI 80	Lo que hace ZTKMONS en cada vuelta	L03
Que el kernel te avise (inotify)	ENF 70 : JES2 avisa SELECT, DESELECT, CHANGE y PURGE	La versión por eventos	L08
<code>lsof / flock</code>	ENQ sobre SYSDSN y GQSCAN	"¿Quién tiene el archivo?" (FDCICS02)	L09
Ser root	APF + estado supervisor / clave 0–7	Lo pide ENF 70	L08

Fuente: Equivalencias: criterio propio para orientarse · servicios según Assembler Services Guide, Using the Subsystem Interface y Authorized Assembler Services Reference (IBM)

Lámina L00-03_unix_vs_zos

📖 Analogía: la caja del supermercado

Los **iniciadores** son las cajas donde se atiende a los jobs. Un **job largo** es un cliente que se queda en la caja todo el día: mientras tanto, esa caja no atiende a nadie más. Un **STC** es un empleado de planta con su propio puesto: no ocupa ninguna caja, y el gerente (el operador) le puede hablar o mandarlo a su casa.

	Job batch largo (ZTKMONS v0, hoy)	STC (ZTKMON, versión final)
Cómo arranca	Se somete como cualquier job (SUBMIT)	El operador: S ZTKMON (o el arranque del sistema)
De dónde sale el JCL	Del miembro que sometés	De una PROC en la PROCLIB
Iniciador	Ocupa uno de su clase todo el tiempo	No usa iniciador batch : tiene su propio address space
Cómo se para	Termina solo (PARM de minutos) o hay que cancelarlo	P ZTKMON : para ordenado , cierra y avisa
Órdenes en vivo	No	F ZTKMON, ESTADO y otras
Quién lo autoriza	Nadie: alcanza con tu librería PPCE.ZTK	Sistemas : alta de la PROC, usuario (perfil STARTED) y APF si usa ENF 70
Para qué lo usamos	Aprender y validar (L04)	El monitor de verdad (L07, L08)

Fuente: Lección L00, sección 2

Lámina L00-04_job_largo_vs_stc

3. Por qué no alcanza con "preguntar cada tanto"

En el análisis de IDEA-005 (2026-09-30) se midió cuánto cuesta preguntarle a JES desde distintos lugares:

Vía	Costo por consulta	Evidencia
SDSF desde REXX (ISFEXEC ST)	9 a 11 s, porque recorre todo el spool	Cabecera de PPCE.ZTK.ASM(ZTKSSI) , DESA 2026-09-03
FTP-JES desde la PC	~0,3 s del lado de JES; el resto es el arranque de la CLI	3 corridas sobre PROD, 2026-09-30
SSI 80 desde Assembler	sin medir : lo mide la práctica del módulo 4	—

Conclusión de entonces: el costo lo ponen **la interfaz de JES y lanzar un job por consulta**, no el lenguaje. Por eso el diseño tiene tres niveles:

Nivel	Cómo funciona	Ventaja	Precio
-------	---------------	---------	--------

A. Polling desde la PC	FTP-JES cada 60 s	Nada en el host	Sin paso ni CC precisos; latencia de 60 s
B. Un job o STC que duerme	Bucle <code>STIMER WAIT</code> + SSI 80 cada N s; manda sólo los cambios	Hora de inicio exacta y CC; sin APF	Ocupa un iniciador (si es job) y consulta aunque no pase nada
C. Por eventos	Listen exit de ENF 70 , que hace POST del ECB; la tarea principal espera en WAIT	Aviso instantáneo y cero consultas en vano	APF + STC: trámite con sistemas

Construimos B primero (ZTKMONS) y C al final. B ya sirve, B valida que entendemos lo que dice JES, y C reusa casi todo B: cambia sólo *quién* despierta al bucle.

Cuánto cuesta preguntarle a JES (lo medido)

L00 · Qué es un demonio en z/OS

Vía	Costo por consulta	Evidencia
SDSF desde REXX (ISFEXEC ST)	9 a 11 s : recorre todo el spool	Cabecera de PPCE.ZTK.ASM(ZTKSSI) , DESA, 2026-09-03
FTP-JES desde la PC	~0,3 s de JES (3,42–3,46 s en total; 3,16 s son el arranque de la CLI de zowe)	3 corridas sobre PROD, 144 filas, 2026-09-30 ~08:50
SSI 80 desde Assembler (ZTKMONS)	sin medir	Lo mide la práctica del módulo 4 (CPU del JESMSG LG ÷ cantidad de vueltas)

Analogía: buscar un libro

SDSF desde REXX es como recorrer **toda la biblioteca** estante por estante para saber qué libros están prestados. SSI 80 es preguntarle directamente al **bibliotecario**, que tiene la lista.

Conclusión: el costo lo ponen **la interfaz de JES** y **lanzar un job por consulta**, no el lenguaje.

Fuente: IDEA-005, tabla "Medido hoy" (2026-09-30)

Lámina L00-06_costo_de_preguntar

📖 Analogía: saber quién entra al edificio

A. Llamás por teléfono a la portería cada minuto y preguntás "¿entró alguien?". **B.** Un vigilante adentro hace una ronda cada 5 segundos y anota sólo lo nuevo. **C.** Ponés un **timbre** en la puerta: suena justo cuando alguien entra, y nadie hace rondas en vano.

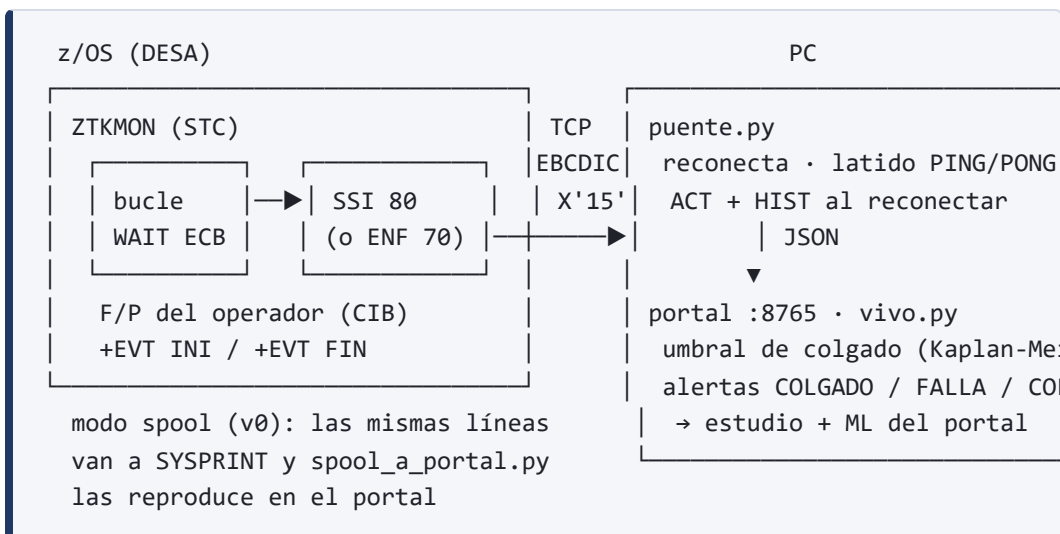
Nivel	Cómo funciona	Ventaja	Precio
A · Preguntar desde la PC	FTP a JES cada 60 s	No hay que tocar nada en el host	Sin paso ni CC precisos; hasta 60 s de atraso
B · Programa en el host que duerme y pregunta	Bucle: STIMER (dormir N s) → SSI 80 → mandar sólo los cambios	Hora de inicio exacta y CC. Sin APF	Pregunta aunque no pase nada; si es job, ocupa un iniciador
C · Por eventos	JES2 avisa por ENF 70 → un exit hace POST del ECB → el bucle despierta con WAIT	Aviso instantáneo y cero consultas en vano	APF + STC: trámite con sistemas

Plan: B primero (ZTKMONS) y C al final. B ya sirve por sí solo, obliga a entender lo que dice JES, y C reusa casi todo B: sólo cambia **quién despierta al bucle**. En B lo despierta el timer; en C, JES.

Fuente: IDEAS_cierres_produccion.md, IDEA-005 (2026-09-30)

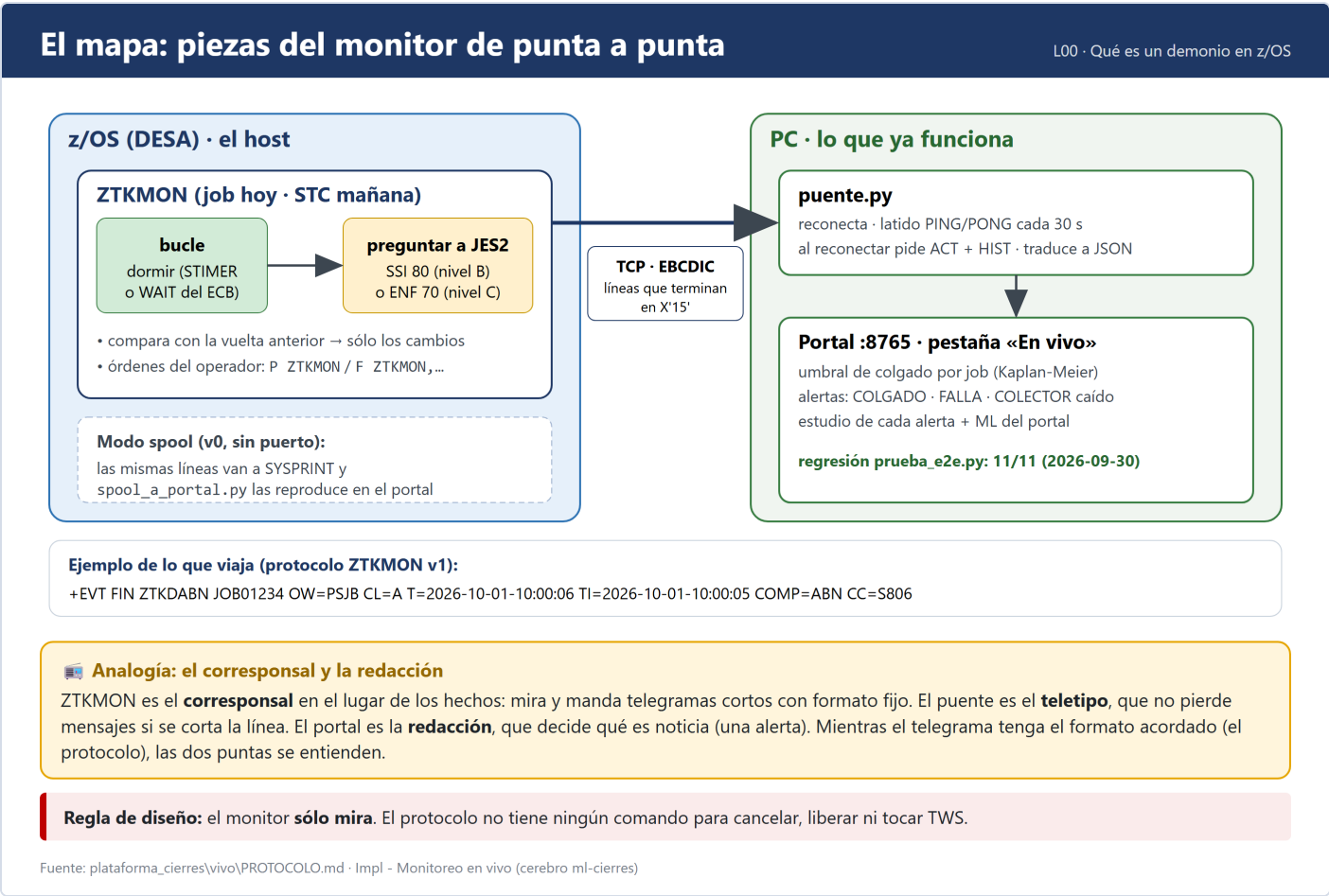
Lámina L00-05_tres_niveles

4. Las piezas del monitor



- El **contrato** entre los dos lados es `PROTOCOLO.md` (ZTKMON v1). Todo lo que funcione contra el simulador funciona contra el host si el host cumple el contrato.

- El monitor **sólo mira**: no existe ningún comando que cambie algo en el host.



1. ¿TIME DEC,...,DATETYPE=YYYYMMDD deja la fecha en los bytes 8-11?
2. ¿Un job que está ejecutando figura con STTRPHAZ = STAT_ONMAIN (14)?
3. ¿Un error de JCL aparece en STAT_POSTEX?
4. ¿STATVRBO contesta desde un programa no autorizado y trae STVBXTST / STVBXTET?
5. ¿Cuánta CPU gasta cada vuelta?

Dónde estamos (2026-10-05)

L00 · Qué es un demonio en z/OS

Pieza	Estado	Evidencia
Protocolo ZTKMON v1	✓ escrito	vivo\PROTOCOLO.md
Puente, simulador, pestaña «En vivo»	✓ andando	prueba_e2e.py 11/11 (2026-09-30)
ZTKMONS v0 (nivel B, modo spool, sin APF)	⚠ escrito, nunca ensamblado	vivo\host\ZTKMONS.asm, 944 líneas
Socket, STC, ENF 70, GQSCAN	🕒 por hacer	Módulos 6 a 9

Las 5 preguntas que contesta la primera práctica real (módulo 4)

#	Pregunta	Qué línea del monitor lo muestra
1	¿TIME DEC,...,DATETYPE=YYYYMMDD deja la fecha en los bytes 8-11?	La fecha del +HOLA
2	¿Un job ejecutando figura con STTRPHAZ = STAT_ONMAIN (14)?	Sí no, salen +DIAG FASE y ningún INI
3	¿Un error de JCL aparece en STAT_POSTEX?	El +EVT FIN de ZTKDJCL
4	¿STATVRBO contesta sin APF y trae STVBXTST / STVBXTET?	Los TI= de cada FIN
5	¿Cuánta CPU gasta cada vuelta?	JESMSGLG de ZTKMONR ÷ 240 vueltas

Fuente: vivo\host\LEEME.md · L00, sección 5

Lámina L00-08_donde_estamos

6. Preguntas de comprobación

Contestalas con tus palabras; no hace falta el detalle técnico todavía.

1. ¿Por qué un demonio que hace while(1){ preguntar(); } sin dormir es un mal demonio? ¿Qué recurso del sistema gasta?
2. ¿Qué diferencia práctica hay entre que ZTKMON sea un **job batch largo** y que sea un **STC**? Pensá en los iniciadores y en cómo lo para el operador.
3. En el nivel C, ¿quién despierta al bucle principal y con qué instrucción? ¿Y en el nivel B?
4. ¿Por qué el protocolo no tiene ningún comando para cancelar un job, aunque técnicamente se podría?

5. Si el puente de la PC se cae 10 minutos y un job arranca **y termina** en ese lapso, ¿cómo se entera el portal? (Pista: `PROTOCOLO.md`.)

Mis respuestas

Relacionado: [Ruta de aprendizaje](#) · [Glosario](#) · siguiente: [L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS](#).

Fuente: IDEAS_cierres_produccion.md (IDEA-005, mediciones del 2026-09-30) · plataforma_cierres\vivo\PROTOCOLO.md · vivo\host\LEEME.md · manuales de IBM citados en la sección 2.

L01 — Anatomía de un programa

Assembler: el prólogo de ZTKMONS

Objetivo: entender línea por línea **cómo entra y cómo sale** un programa de z/OS, cómo recibe el `PARM` y por qué ZTKMONS es `AMODE 31 RMODE 24`. Todo lo que viene después (el bucle, el SSI, el socket) vive entre este prólogo y este epílogo.

Usamos el código real: `plataforma_cierres\vivo\host\ZTKMONS.asm`. Los números de línea son de ese archivo.

1. Cómo llega el control a tu programa

Cuando el sistema (o otro programa) te llama, **los registros traen información por convención:**

Registro	Qué trae al entrar	Para qué lo usás
R1	Dirección de la lista de parámetros	Leer el <code>PARM</code> del <code>EXEC</code>
R13	Dirección de la save area del que te llamó (18 palabras = 72 bytes)	Guardar ahí sus registros
R14	Dirección de retorno	Volver al terminar
R15	Tu dirección de entrada	Usarla como base

Y al salir **vos** tenés que dejar:

Registro	Qué dejás al salir
R0–R14	Los mismos valores que te dieron (los restaurás)
R15	El return code . Es el <code>COND CODE</code> que ves en la bitácora

 Fuente: *z/OS MVS Programming: Assembler Services Guide*, capítulo *Linkage conventions* (convención estándar con save area de 72 bytes).

🌟 Analogía: la posta del relevo

Cuando te pasan la posta, el que te la da te deja cuatro cosas en la mano: **R1** = la mochila con el encargo (el PARM), **R13** = su libreta, donde tenés que anotar cómo te dejó las cosas, **R14** = el lugar adonde tenés que volver, y **R15** = tu propio número de corredor. Al volver le devolvés todo como estaba, más **una nota en R15**: cómo te fue (el RC).

Al ENTRAR (te lo dan)

R1	Dirección de la lista de parámetros (el PARM)
R13	Dirección de la save area del que te llamó (72 bytes)
R14	Dirección de retorno
R15	Tu dirección de entrada → sirve de base

Al SALIR (lo dejás vos)

R0-R14	Igual a como te los dieron (los restaurás)
R15	El return code : es el COND CODE que ves en la bitácora (0000, 0004, 0008...)

Si al salir R15 vale 8, en el JESMSGGLG ves \$HASP395 ... ENDED - RC=0008 .

Fuente: z/OS MVS Programming: Assembler Services Guide, capítulo "Linkage conventions"

Lámina L01-01_registros_al_entrar_y_salir

2. La save area: la "pila" del mundo z/OS

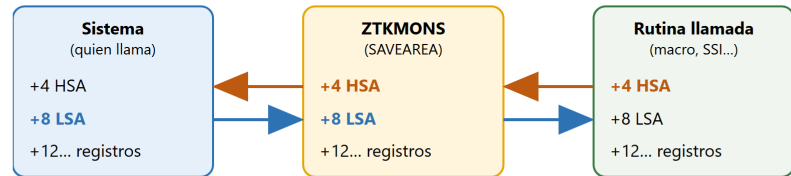
z/OS no usa una pila de hardware. Cada programa tiene un bloque de 18 palabras y los bloques **se encadenan**:

Offset	Contenido
0	(uso de lenguajes, ignorado)
4	HSA : puntero a la save area <i>anterior</i> (la de quien te llamó)
8	LSA : puntero a la save area <i>siguiente</i> (la de a quien llamás)
12	R14
16	R15
20 ... 68	R0, R1, ..., R12

Es una lista doblemente enlazada. Gracias a ella, un dump puede reconstruir quién llamó a quién.

Offset	Contenido (18 palabras = 72 bytes)
0	uso de lenguajes (se ignora)
4	HSA → save area anterior
8	LSA → save area siguiente
12	R14
16	R15
20	R0
24	R1
28 ... 64	R2 ... R11
68	R12

Se encadenan como vagones (lista doble):



azul = LSA (hacia adelante: a quién llamé)

naranja = HSA (hacia atrás: quién me llamó)

Gracias a esta cadena, un dump reconstruye quién llamó a quién, aunque no haya una pila de hardware.

🚂 Analogía: los vagones de un tren

Cada programa engancha su vagón (su save area) al del que lo llamó: un gancho mira hacia atrás (HSA) y otro hacia adelante (LSA). Para volver, cada uno se suelta y vuelve al vagón de atrás.

Fuente: z/OS MVS Programming: Assembler Services Guide, "Linkage conventions" (formato de la save area de 72 bytes)

Lámina L01-02_save_area

3. El prólogo de ZTKMONS (líneas 89–101)

```

ZTKMONS  CSECT
          STM  R14,R12,12(R13)    (1) guardo R14..R12 en la save area
          LR   R12,R15             (2) R12 = mi dirección de entrada → p
          LA   R11,4095(,R12)      (3) R11 = R12 + 4095 ...
          LA   R11,1(,R11)         ... + 1 = R12 + 4096 → segunda base
          LA   R10,4095(,R11)      (4) R10 = R11 + 4096 → tercera base
          LA   R10,1(,R10)
          USING ZTKMONS,R12,R11,R10 (5) le aviso al ensamblador cuáles se
          ST   R1,SALVAR1          (6) guardo el puntero al PARM para de
          LA   R2,SAVEAREA         (7) R2 = MI save area
          ST   R13,4(,R2)          (8) mi HSA = la del que me llamó
          ST   R2,8(,R13)          (9) su LSA = la mía
          LR   R13,R2              (10) desde ahora R13 apunta a MI save
    
```

Por qué cada paso:

- **(1)** `STM R14,R12,12(R13)` guarda 15 registros (R14, R15, R0 ... R12; el rango "da la vuelta") a partir del offset 12. **No toca R13**: ese lo guardamos aparte en (8).
- **(2)–(5) Registros base.** Una instrucción RX/RS direcciona con base + desplazamiento, y el desplazamiento tiene **12 bits (0–4095)**. Por eso un

registro base cubre 4 KB de código y datos. ZTKMONS es grande, así que usa tres bases contiguas (R12, R11, R10): unos 12 KB direccionables. El `LA ...,4095 + LA ...,1` existe porque `LA` no admite un desplazamiento de 4096.

- **USING** no genera código: es una **promesa al ensamblador**: "en ejecución, R12 va a valer la dirección de ZTKMONS". Si la promesa es falsa, el programa direcciona basura.
- **(6)** R1 se pisa enseguida (macros como OPEN y WTO lo usan), por eso se guarda.
- **(7)–(10)** Encadenamiento. Después de (10) ya podés llamar a otros (macros del sistema, SSI) sin romper la save area del que te llamó.

 Fuente: *HLASM Language Reference* (instrucción USING y direccionamiento base-desplazamiento) · *z/Architecture Principles of Operation* (formatos RX/RS, desplazamiento de 12 bits).

El prólogo de ZTKMONS, paso a paso (líneas 89–101)

L01 · Anatomía de un programa Assembler

ZTKMONS CSECT

STM R14,R12,12(R13) (1) guardo R14..R12 en la save area DEL QUE ME LLAMÓ

LR R12,R15 (2) R12 = mi dirección de entrada → 1.ª base

LA R11,4095(,R12) (3) R11 = R12 + 4095 ...

LA R11,1(,R11) ... + 1 = R12 + 4096 → 2.ª base

LA R10,4095(,R11) (4) R10 = R11 + 4096 → 3.ª base

LA R10,1(,R10)

USING ZTKMONS,R12,R11,R10 (5) le aviso al ensamblador cuáles son mis bases

ST R1,SALVAR1 (6) guardo el puntero al PARM para después

LA R2,SAVEAREA (7) R2 = MI save area

ST R13,4(,R2) (8) mi HSA = la del que me llamó

ST R2,8(,R13) (9) su LSA = la mía

LR R13,R2 (10) desde ahora R13 apunta a MI save area

Paso	Qué logra	Por qué
(1)	No romperle nada al que me llamó	Guarda 15 registros (R14, R15, R0...R12: el rango "da la vuelta") desde el offset 12. R13 no : va aparte en (8)
(2)–(4)	Tres registros base contiguos	Cada base cubre 4 KB. <code>LA</code> no admite desplazamiento 4096, por eso 4095 + 1
(5)	Que el ensamblador sepa direccionar	<code>USING</code> no genera código : es una promesa sobre lo que valdrán R12, R11 y R10
(6)	No perder el PARM	OPEN, WTO y otras macros pisan R1
(7)–(10)	Enganchar mi vagón a la cadena	Desde acá ya puedo llamar a macros y al SSI sin romper la save area del que me llamó

Fuente: vivo\host\ZTKMONS.asm, líneas 89–101 · HLASM Language Reference (USING)

Lámina L01-03_prologo_paso_a_paso

Memoria del programa (desde el inicio de ZTKMONS) →

R12 cubre +0 ... +4095 (4 KB)	R11 cubre +4096 ... +8191 (R12 + 4096)	R10 cubre +8192 ... +12287 (R11 + 4096)	fuera de las bases
---	--	---	------------------------------

Una instrucción RX/RS direcciona con **base + desplazamiento**, y el desplazamiento tiene **12 bits: 0 a 4095**.

Ejemplo: un dato a +5000 se direcciona con R11 y desplazamiento $5000 - 4096 = 904$.

Las tablas grandes (NUEVA, VIEJA, REPORT) van al final, **fuera** de las bases, y se usan por dirección =A(...).

🔦 **Analogía: linternas en un pasillo oscuro**

Cada registro base es una linterna que ilumina **exactamente 4 metros** (4096 bytes) de pasillo. ZTKMONS mide unos 12 metros de código y datos, así que necesita **tres linternas** puestas una a continuación de la otra. Lo que queda más allá (las tablas grandes) no se ilumina con linterna: se va con la dirección exacta en la mano.

Si la promesa del USING es falsa (por ejemplo, olvidás cargar R11), el ensamblador no se queja, pero en ejecución el programa lee y escribe en lugares equivocados.

Fuente: z/Architecture Principles of Operation (formatos RX/RS, desplazamiento de 12 bits) · ZTKMONS.asm

Lámina L01-04_registros_base

4. El epílogo (líneas 220–224)

SALIR	DS	0H	
	L	R13,4(,R13)	vuelvo a la save area del que me llamo
	L	R14,12(,R13)	recupero su dirección de retorno
	LM	R0,R12,20(R13)	recupero R0..R12
	BR	R14	vuelvo. R15 = return code (NO se resta)

Detalle fino: el epílogo restaura R0–R12 desde el offset 20, **no** R14–R12 desde el offset 12. Si hiciera `LM R14,R12,12(R13)`, pisaría R15 con el valor que traía al entrar, y **el RC se perdería**. Muchos programas usan la macro `RETURN (14,12),RC=(15)`, que hace exactamente esto.

✓ Lo que hace ZTKMONS (líneas 220–224)

```
SALIR DS 0H
L R13,4(,R13)    vuelvo al vagón de atrás
L R14,12(,R13)   mi dirección de retorno
LM R0,R12,20(R13) R0..R12
BR R14           R15 = RC, intacto
```

Restaura todo **menos R15**, que lleva el RC.

✗ El error clásico

```
L R13,4(,R13)
LM R14,R12,12(R13) ← también R15
BR R14
```

Pisa R15 con el valor que traía al **entrar** (la dirección de entrada), y el RC calculado **se pierde**.

Forma	Qué hace con R15
LM R0,R12,20(R13) + L R14,12(,R13)	Lo deja como está → el RC llega a JES
RETURN (14,12),RC=(15)	Lo mismo, en forma de macro
LM R14,R12,12(R13)	Lo pisa con lo que traía al entrar

📦 Analogía: devolver la caja de herramientas

Devolvés cada herramienta a su lugar, pero la **nota con el resultado del trabajo** (R15) la dejás arriba. Si guardás también la nota vieja encima, el dueño lee la nota de ayer.

Fuente: vivo\host\ZTKMONS.asm, líneas 220–224 · Assembler Services Guide (RETURN)

Lámina L01-05_epilogo

5. Cómo llega el PARM (rutina PARSEA, línea 773)

Con `//PASO EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20'`, el sistema arma esto:



```
L R2,SALVAR1      R2 = R1 original (la lista)
L R2,0(,R2)       R2 = dirección del área del PARM
LH R3,0(,R2)      R3 = longitud (media palabra)
LTR R3,R3
BNP PMALO         sin PARM → error
...
LA R2,2(,R2)      R2 = primer carácter del texto
```

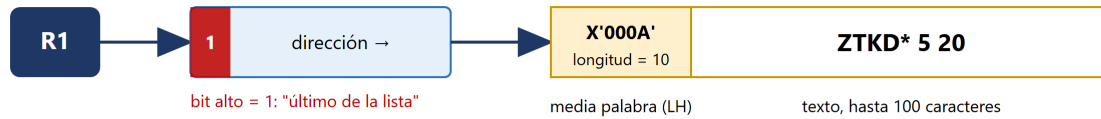
¿Y el bit alto? La palabra trae el bit 0 encendido (marca de fin de lista). En **AMODE 31** la dirección efectiva usa sólo los 31 bits de la derecha, así que el bit se ignora al direccionar. En **AMODE 24** también se ignora (usa 24 bits). **No**

sería inocuo si comparás o sumás esa palabra como número: ahí habría que limpiarlo con `N R2,=X'7FFFFFFF'`.

Cómo llega el PARM (rutina PARSEA, línea 773)

L01 · Anatomía de un programa Assembler

```
//PASO EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20'
```



La 1.ª palabra de la lista trae el bit 0 encendido. En AMODE 31 la dirección efectiva usa los 31 bits de la derecha, así que al **direccionar** el bit se ignora. Si la usás como **número** (comparar, sumar), límpiala: `N R2,=X'7FFFFFFF'`.

L	R2,SALVAR1	R2 = R1 original (la lista)
L	R2,0(R2)	R2 = dirección del área del PARM
LH	R3,0(R2)	R3 = longitud (media palabra)
LTR	R3,R3	
BNP	PMALO	sin PARM (longitud 0) → error
LA	R2,2(R2)	R2 = primer carácter del texto

📌 Analogía: el sobre con la carta

R1 te da la dirección de un **casillero**; en el casillero hay un **sobre**; el sobre dice en el frente cuántas hojas trae (la longitud) y adentro está la carta (el texto). El sello rojo (bit alto) sólo dice "es el último sobre del casillero".

Fuente: vivo\host\ZTKMONS.asm, rutina PARSEA (línea 773) · Assembler Services Guide (lista de parámetros del EXEC PARM)

Lámina L01-06_como_llega_el_parm

6. AMODE 31, RMODE 24 y el DCB (líneas 1–3 y 933–935)

- **AMODE** (*addressing mode*) dice con cuántos bits direcciona el programa al ejecutar: 24 bits = 16 MB, 31 bits = 2 GB.
- **RMODE** (*residence mode*) dice **dónde puede cargarse**. RMODE 24 = debajo de la "línea" de 16 MB.
- ZTKMONS es **RMODE 24** porque su **DCB de QSAM** (`PRTDCB`, para `SYSPRINT`) vive dentro de la CSECT, y QSAM exige el DCB bajo la línea. Lo dice el comentario de la línea 933.
- Es **AMODE 31** porque las áreas que devuelve JES por SSI 80 pueden estar **arriba** de la línea.

Después del OPEN, el programa verifica que abrió:

```

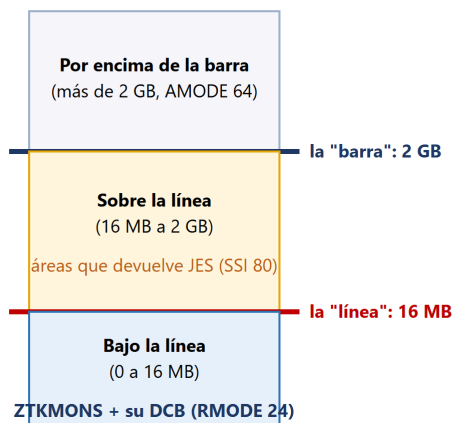
OPEN  (PRTDCB,OUTPUT)
TM    PRTDCB+48,X'10'      DCBOFLGS (offset 48), bit DCBOFOPN =
BO    CONPRT
WTO   'ZTKMONS: FALTA EL DD SYSPRINT'

```

Fuente: z/OS MVS Programming: Assembler Services Guide (AMODE/RMODE) · z/OS DFSMS Macro Instructions for Data Sets (DCB, campo DCBOFLGS y bit de OPEN exitoso).

AMODE, RMODE y por qué el DCB vive "abajo"

L01 · Anatomía de un programa Assembler



	Qué significa	En ZTKMONS
AMODE	Con cuántos bits direcciona al ejecutar: 24 → 16 MB · 31 → 2 GB · 64 → arriba de la barra	31 : las áreas de JES pueden estar sobre la línea
RMODE	Dónde puede cargarse el programa: 24 = bajo la línea; ANY = donde sea	24 : el DCB está dentro de la CSECT
DCB de QSAM	Describe el archivo (SYSPRINT). QSAM exige que esté bajo la línea	PRTDCB, línea 934
¿Abrió?	Después del OPEN se mira DCBOFLGS (offset 48), bit X'10'	TM PRTDCB+48,X'10'

⚡ Analogía: un edificio con un ascensor viejo

La memoria es un edificio. QSAM es un empleado antiguo que **no sube escaleras**: sólo atiende en la **planta baja** (bajo la línea). Por eso el DCB, que es el formulario que él tiene que leer, debe estar en planta baja (RMODE 24). En cambio, el programa puede **visitar** cualquier piso hasta el 2 GB (AMODE 31), porque JES le deja las respuestas en pisos altos.

Fuente: Assembler Services Guide (AMODE/RMODE) · z/OS DFSMS Macro Instructions for Data Sets (DCB, DCBOFLGS) · ZTKMONS.asm líneas 1–3 y 933–935

Lámina L01-07_amode_rmode

7. Dos decisiones de diseño que vamos a revisar más adelante

Decisión en v0	Por qué está bien hoy	Cuándo cambia
La save area y los datos están dentro de la CSECT (SAVEAREA DS 18F, línea 930) → el programa no es reentrante	Un job que corre una sola copia no lo necesita	L07: un STC con varias tareas o en LPA debería ser reentrante (GETMAIN del área de trabajo)

Las tablas grandes (NUEVA, VIEJA, REPORT) van al final , fuera de las tres bases , y se usan por dirección =A(...)	Si estuvieran en medio, harían falta más registros base	Si el programa crece más de 12 KB de código, falta otra base o hay que partirlo
--	---	---

Dos decisiones de v0 que vamos a revisar

L01 · Anatomía de un programa Assembler

Decisión en ZTKMONS v0	Por qué está bien hoy	Cuándo cambia
Save area y datos dentro de la CSECT (SAVEAREA DS 18F, línea 930) → no es reentrante	Un job que corre una sola copia no lo necesita	L07: un STC con varias tareas o cargado en LPA debería ser reentrante (área de trabajo con GETMAIN / STORAGE)
Tablas grandes (NUEVA, VIEJA, REPORT) al final, fuera de las bases, usadas por =A(...)	Si estuvieran en medio, harían falta más registros base	Si el código pasa los 12 KB, falta otra base o hay que partir el programa

Analogía: el formulario pegado en la pared

Un programa **no reentrante** es un formulario pegado en la pared donde cada uno escribe encima: sirve si lo usa una sola persona. Uno **reentrante** es un formulario modelo del que cada uno saca **su propia fotocopia** (su área de trabajo) para escribir.

Fuente: vivo\host\ZTKMONS.asm, líneas 925-944

Lámina L01-08_decisiones_de_diseno

8. Preguntas de comprobación

1. Si te olvidás de la línea (9) (ST R2,8(R13)), ¿el programa falla? ¿Qué es lo que sí se pierde?
2. ¿Por qué el epílogo usa LM R0,R12,20(R13) y no LM R14,R12,12(R13)? ¿Qué vería la bitácora si se usara el segundo?
3. Una instrucción está a 9.000 bytes del inicio de ZTKMONS. ¿Con qué registro base la direcciona el ensamblador y con qué desplazamiento?
4. Si corrés ZTKMONS **sin** PARM, ¿qué valor tiene R3 después del LH y por qué rama sale?
5. ¿Qué pasaría si ZTKMONS fuera RMODE ANY?

9. Práctica en DESA (la escribís vos)

1. Abra SYSPRINT (LRECL 133) y verifique el OPEN con el **TM** de la sección 6.
2. Escriba una línea **+HOLA PARM=<texto del PARM>**.
3. Termine con **RC = 4** si vino PARM y **RC = 8** si no vino.

- el JOBID de cada corrida;
- el RC que da \$HASP395 ... ENDED - RC= en el JESMSG LG (tiene que dar 0004 y 0008);
- el SYSPRINT.

Si el ensamblado da errores, traé el listado de ASMA90: **leer esos errores es parte de la lección**. Lo archivamos en `20 - Practicas` como `P01 - ZTKHOLA`.

Pista: para escribir una línea, mirá cómo lo hace la rutina `MANDA` de `ZTKMONS` (`PUT PRTDCB,SALIDA`).

Práctica P01: ZTKHOLA (la escribís vos)

L01 · Anatomía de un programa Assembler

Paso	Qué tiene que hacer el programa	Qué reusás de ZTKMONS
1	Mismo prólogo : STM, base, USING, guardar R1 y encadenar la save area	Líneas 89–101
2	Abrir SYSPRINT (LRECL 133) y verificar el OPEN	<code>OPEN + TM ...+48,X'10'</code>
3	Escribir <code>+HOLA PARM=<texto del PARM></code>	Rutina <code>MANDA</code> (<code>PUT PRTDCB,SALIDA</code>) y <code>PARSEA</code>
4	<code>RC = 4</code> si vino PARM y <code>8</code> si no vino	<code>LA R15,...</code> antes del epílogo
5	Mismo epílogo , sin pisar R15	Líneas 220–224

Corrida	PARM	Resultado esperado	Qué traés
A	<code>PARM='PRUEBA'</code>	<code>\$HASP395 ... RC=0004</code> y <code>+HOLA PARM=PRUEBA</code>	JOBID, la línea <code>\$HASP395</code> y el SYSPRINT. Si falla el ensamblado, el listado de ASMA90
B	(sin PARM)	<code>\$HASP395 ... RC=0008</code>	

Leer los errores de ASMA90 **es parte de la lección**: si algo no ensambla, mejor.

Fuente: Lección L01, sección 9

Lámina L01-09_practica_ztkhola

Relacionado: [L00 - Que es un demonio en z-OS y que vamos a construir](#) · [Glosario](#) · [Ruta de aprendizaje](#) · siguiente: [L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB](#).

 Fuente: código `vivo\host\ZTKMONS.asm` (v0, 944 líneas) y los manuales de IBM citados en cada sección.

L02 — Dormir sin gastar CPU: STIMER, WAIT, POST y ECB

Objetivo: escribir el corazón de cualquier demonio, un **bucle que duerme** y despierta cuando corresponde, y **medir** que dormido no gasta CPU. Al final sabés combinar en una sola espera el timer, las órdenes del operador y los eventos.

1. El bucle

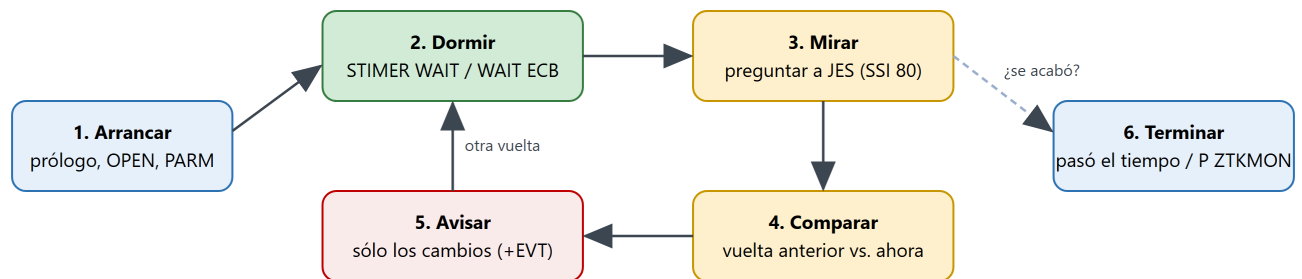
Todas las versiones del monitor tienen el mismo bucle de seis pasos:

1. **Arrancar:** prólogo, OPEN, leer el PARM (ya lo viste en L01).
2. **Dormir** hasta que pase el intervalo o alguien te despierte.
3. **Mirar:** preguntarle a JES (L03).
4. **Comparar** con la vuelta anterior.
5. **Avisar sólo los cambios** (+EVT INI / +EVT FIN).
6. **Terminar** cuando se cumple el tiempo o llega un P ZTKMON.

El paso 2 es el que hace que el demonio sea bueno o malo. En ZTKMONS v0 está en la línea 199:

NOLAT	DS	0H	
	STIMER	WAIT,BINTVL=INTBIN	duermo el intervalo del PARM
	L	R1,ITER	cuento la vuelta
	BCTR	R1,0	
	ST	R1,ITER	
	LTR	R1,R1	
	BP	BUCLE	quedan vueltas → otra vez

Lo único que cambia entre el nivel B y el C (L00) es **quién despierta al bucle** en el paso 2: en B lo despierta el reloj, en C lo despierta JES.



🦊 Analogía: el guardia nocturno con despertador

Pone el despertador (2), se despierta y mira las cámaras (3), compara con lo que vio la última vez (4), anota sólo lo nuevo (5) y vuelve a poner el despertador. Casi toda la noche está en el paso 2: **durmiendo, sin gastar**.

Este bucle es el mismo en **todas** las versiones. Lo único que cambia entre el nivel B y el C es **quién lo despierta** en el paso 2: el reloj o JES.

Fuente: ZTKMONS.asm (bucle principal) · criterio propio para el diagrama

Lámina L02-01_el_bucle

2. STIMER: dormir un intervalo

Forma	Qué hace	Unidad
STIMER WAIT, BINTVL=campo	La tarea queda en espera hasta que vence el intervalo	Palabra binaria en centésimas de segundo (500 = 5 s)
STIMER WAIT, DINTVL=campo	Igual, con el intervalo en decimal	8 bytes empaquetados HHMMSSth
STIMER REAL, rutina, BINTVL=...	No espera : el programa sigue y, al vencer, el sistema llama a la rutina de salida	Igual
STIMERM SET, ID=..., BINTVL=..., EXIT=...	Como REAL, pero con un identificador : se puede tener más de uno y cancelarlo con STIMERM CANCEL, ID=...	Igual

En ZTKMONS, `INTBIN` se calcula a partir del `PARM`: segundos × 100.

El límite de STIMER WAIT: mientras duerme, **no puede atender nada más**. Si el operador manda `P ZTKMON` en el segundo 1 de un intervalo de 60 s, el

programa se entera recién en el segundo 60. Para un job de prueba alcanza; para el STC no. Por eso existe la sección 4.

 Fuente: *z/OS MVS Programming: Assembler Services Reference* (STIMER, STIMERM) · *Assembler Services Guide*, capítulo de servicios de tiempo · `ZTKMONS.asm` línea 199.

STIMER: dormir un intervalo

L02 · Dormir sin gastar CPU

Forma	Qué hace	Unidad del intervalo
STIMER WAIT,BINTVL=campo	La tarea queda en espera hasta que vence el intervalo	Palabra binaria en centésimas de segundo (500 = 5 s)
STIMER WAIT,DINTVL=campo	Igual, con el intervalo en decimal	8 bytes empaquetados HHMMSSth
STIMER REAL,rutina,BINTVL=...	No espera: sigue trabajando y, al vencer, el sistema llama a una rutina de salida	Igual que arriba
STIMERM SET,ID=...,EXIT=...	Como REAL pero con varios timers identificados; se cancela con STIMERM CANCEL	BINTVL / DINTVL / ...

```
SEG5    DC    F'500'           5 segundos = 500 centésimas
...
VUELTA  DS    0H
        STIMER WAIT,BINTVL=SEG5 duermo 5 s (CPU = 0)
        BAL   R14,MIRAR        pregunto a JES
        B     VUELTA
```

 **Analogía: el temporizador de cocina**

STIMER WAIT es poner el temporizador y **sentarse a esperar** que suene. STIMER REAL es ponerlo y **seguir cocinando**: cuando suena, alguien (la rutina de salida) te toca el hombro.

Con STIMER WAIT el bucle **no puede atender otra cosa** mientras duerme (por ejemplo, un P ZTKMON). Para eso se combina timer + ECB (siguiente lámina).

Fuente: *z/OS MVS Programming: Assembler Services Reference* (STIMER, STIMERM) · *Assembler Services Guide* (timing services)

Lámina L02-02_stimer

3. El ECB: un semáforo de una palabra

Un **ECB** (*event control block*) es una palabra (4 bytes) alineada a palabra:

Bits	Significado
bit 0 (X'80' en el primer byte)	"Alguien está esperando": lo pone WAIT
bit 1 (X'40')	"Completo": lo pone POST, y eso despierta al que espera
bits 2–31	Código de finalización : lo que mandó el POST

Instrucción	Qué hace	En Unix
XC ECB,ECB	Limpia el ECB antes de esperar	reiniciar el semáforo
WAIT ECB=ECB	Duerme hasta que el ECB esté completo	sem_wait()
POST ECB,código	Lo marca completo y despierta al que espera	sem_post()
WAIT 1,ECBLIST=LISTA	Duerme hasta que uno de varios ECB esté completo	select()

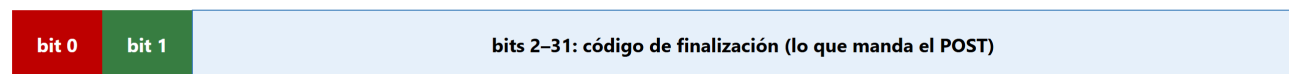
Trampa clásica: si no limpiás el ECB después de atenderlo, sigue con el bit "completo" encendido y el próximo WAIT **pasa de largo**. El bucle gira sin dormir y vuelve a ser un mal demonio.

 Fuente: *Assembler Services Guide*, "Synchronizing tasks" (WAIT, POST y el formato del ECB).

El ECB: un semáforo de una palabra

L02 · Dormir sin gastar CPU

ECB = 1 palabra (4 bytes, 32 bits), alineada a palabra



X'80' = "alguien espera" (lo pone WAIT)

X'40' = "completo" (lo pone POST): el que espera se despierta

Instrucción	Qué hace	Equivalente Unix
XC ECB,ECB	Limpia el ECB antes de esperar (si quedó el X'40', el WAIT pasa de largo)	reiniciar el semáforo
WAIT ECB=ECB	La tarea duerme hasta que el ECB esté completo	sem_wait()
POST ECB,código	Marca completo (X'40') + código, y despierta al que espera	sem_post()
WAIT 1,ECBLIST=LISTA	Duerme hasta que uno de varios ECB esté completo	select()

Analogía: la banderita del buzón

El ECB es la **banderita** de un buzón de campo. Bajás la banderita (XC) y te vas a dormir (WAIT). Cuando el cartero deja algo, **levanta la banderita** (POST) y eso te despierta. El "código" es la nota que dejó: qué tipo de correspondencia trajo.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Synchronizing tasks": WAIT, POST y formato del ECB)

Lámina L02-03_ecb

4. Timer + operador + eventos en una sola espera

La forma de que el bucle atienda todo sin demora es esperar en **una lista de ECB**:

ECB	Quién lo postea	Cuándo
ECBTIEMPO	La rutina de salida de STIMERM SET	Al vencer el intervalo
El del COM (COMECBPT, L07)	El sistema	Cuando el operador manda F o P
ECBEVENTO	El listen exit de ENF 70 (L08)	Cuando un job cambia de estado

LISTA	DC	A(ECBTIEMPO)	1. ^a : dirección del ECB del timer
	DC	A(0)	2. ^a : dirección del ECB del COMM (se
	DC	A(ECBEVENTO)	3. ^a y ÚLTIMA: dirección del ECB de e
...			
	OI	LISTA+8,X'80'	bit alto en la ÚLTIMA entrada = fin
	WAIT	1,ECBLIST=LISTA	duermo hasta que 1 de los 3 esté cor

Al despertar se mira **cuál** se posteó (bit X'40' de cada ECB), se atiende sólo eso, se limpia **ese** ECB y se vuelve a esperar.

La rutina de salida del timer corre de forma **asíncrona**, en la misma tarea, mientras el programa principal está en WAIT. Tiene que ser **corta**: hacer POST ECBTIEMPO y volver. La forma exacta de su enlace (qué registros recibe y cómo vuelve) se escribe y se prueba en la práctica.

 Fuente: *Assembler Services Guide* (WAIT con ECBLIST; STIMERM con rutina de salida).
Diseño de la lista: criterio propio.



LISTA	DC	A(ECBTIEMPO)	1.ª: dirección del ECB del timer
	DC	A(0)	2.ª: dirección del ECB del COMM (COMECBPT, se llena en L07)
	DC	A(ECBEVENTO)	3.ª y ÚLTIMA: dirección del ECB de eventos
...			
	OI	LISTA+8,X'80'	bit alto encendido en la ÚLTIMA entrada = fin de la lista
	WAIT	1,ECBLIST=LISTA	duermo hasta que 1 de los 3 esté completo

🔔 Analogía: la recepción de un hotel con tres timbres

El recepcionista duerme en su silla. Hay tres timbres: el reloj despertador, el teléfono del gerente y el timbre de la puerta. Se despierta con **el primero que suena**, mira cuál fue y atiende sólo eso.

La forma exacta de armar la lista y la rutina de salida del timer se escribe y prueba en la práctica de L02.

Fuente: Assembler Services Guide (WAIT con ECBLIST, STIMERM con rutina de salida) - criterio propio para el diseño

Lámina L02-04_timer_y_eventos_juntos

5. Cómo se mide que "dormido no gasta"

Al terminar un paso, el sistema escribe su consumo en el JESYSMSG y en el resumen del JESMSGGLG:

- IEF032I STEP/... /STOP ... CPU: 0 HR 00 MIN 00.01 SEC → CPU del paso.
- La columna **CPU** del resumen por paso del JESMSGGLG (la misma que lee la bitácora).

La cuenta es **CPU total ÷ cantidad de vueltas = costo de despertar una vez**. Ese número es la base para medir después lo que agrega SSI 80 (L04) y para decidir el intervalo en producción (L11).

6. Preguntas de comprobación

1. ¿Qué pasa con la CPU si te olvidás el `XC ECB,ECB` después de atender un `POST`? ¿Por qué?
2. ¿Cuántas centésimas pondrías en `BINTVL` para dormir 2,5 segundos?

- ## Mis respuestas

Partí de tu ZTKHOLA (P01):

- Monitor z · cuaderno de estudio

Traé:

- el JOBID de cada corrida;
- la CPU del paso (IEF032I);
- la diferencia de hora entre dos líneas seguidas del SYSPRINT.

El número que dé **no lo sabemos todavía**: es justamente lo que mide la práctica.

Extra (opcional): reemplazá STIMER WAIT por STIMERM SET + WAIT sobre un ECB que postea la rutina de salida. Es el primer paso hacia la ECBLIST del STC.

Práctica P02: el bucle y su CPU

L02 · Dormir sin gastar CPU

Paso	Qué hace el programa ZTKBUCLE	Qué aprendés
1	PARM 'segundos minutos' , por ejemplo '5 2'	Reusar PARSEA (L01)
2	En cada vuelta: STIMER WAIT y escribir la hora (TIME DEC) en SYSPRINT	El bucle y el reloj
3	Termina a los M minutos, con la cantidad de vueltas	Cortar el bucle
4	Correr con '5 2' (24 vueltas) y con '1 2' (120 vueltas)	Comparar la CPU

Qué mirar en el JESMSG LG / JESYSMSG	Qué esperamos
CPU del paso (IEF032I ... CPU: o la columna CPU del resumen)	Muy baja: centésimas de segundo, aunque el job dure 2 minutos
Hora entre líneas del SYSPRINT	Muy cerca del intervalo pedido
CPU ÷ vueltas	El costo de "despertar y escribir". Es la base para medir después el SSI 80 (L04)

El número que salga **no lo sabemos todavía**: es justamente lo que mide la práctica.

Fuente: Ruta de aprendizaje, módulo 2

Lámina L02-05_practica_medir_cpu

Relacionado: [L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS](#) · [Glosario](#) · siguiente: [L03 - Hablarle a JES por SSI 80](#).

 Fuente: vivo\host\ZTKMONS.asm (bucle, líneas 190–206; AHORA, línea 234) y los manuales de IBM citados en cada sección.

L03 — Hablarle a JES por SSI 80

Objetivo: entender cómo un programa le pregunta a JES2 qué jobs están ejecutando y cuáles terminaron, cómo se recorre la respuesta y por qué hay que **devolver la memoria**. Es lo que hace ZTKMONS en cada vuelta.

1. Qué es el SSI

El **SSI** (*Subsystem Interface*) es la ventanilla por la que un programa le pide cosas a un subsistema, en nuestro caso JES2. Cada trámite tiene un **código de función**; el **80** es *extended status*: el estado de los jobs, con mucho más detalle que el viejo "status".

Tres datos importantes:

- Se llama con la macro `IEFSSREQ`, con R1 apuntando a una palabra que contiene la dirección del SSOB.
- La macro necesita los mapeos **CVT** e **IEFJESCT**, porque busca la rutina del SSI a partir del CVT. Por eso ZTKMONS los incluye al principio (línea 73).
- **No requiere APF**. ZTKJACT lo usa desde `PPCE.ZTK.LOAD` con AC=0, en estado problema.

 Fuente: *z/OS MVS Using the Subsystem Interface* (función 80, *extended status*) · cabecera de `ZTKMONS.asm`.



Y al terminar de leerla: otro pedido con STATTYPE = STATMEM para DEVOLVER esa memoria.

🏠 Analogía: la ventanilla del banco

El **SSOB** es el formulario de trámite ("quiero consultar estados", función 80), el **SSIB** dice a qué ventanilla va (JES2) y el **STAT** es la hoja con el detalle de la consulta (qué jobs, qué fase). JES te devuelve una **carpeta con fichas encadenadas**, una por job. Cuando terminás de leerlas, **devolvés la carpeta** (STATMEM); si no, se acumulan en tu escritorio (memoria perdida).

Fuente: z/OS MVS Using the Subsystem Interface (SSI function code 80, extended status) · mapeo IAZSSST · ZTKMONS.asm (LLAMASSI)

Lámina L03-01_ventanilla_ssi

2. Armar el pedido: SSOB, SSIB y STAT

Bloque	Mapeo	Para qué
SSOB	IEFSSOBH	El "formulario": qué función se pide y dónde están los otros bloques
SSIB	IEFJSSIB	A qué subsistema va: JES2
STAT	IAZSSST	El detalle de la consulta de la función 80: qué jobs, qué fase y qué nivel de detalle

Así lo arma ZTKMONS (rutina INICIA, líneas 261–285):

```

MVC  QSSOB.SSOBID,=C'SSOB'
MVC  QSSOB.SSOBLEN,=Y(SSOBHSIZ)
MVC  QSSOB.SSOBFUNC,=Y(SSOBESTA)    función 80: extended statu
ST   R3,QSSOB.SSOBSSIB              → SSIB
ST   R4,QSSOB.SSOBINDV              → STAT (el área de la func
MVC  QSSIB.SSIBID,=C'SSIB'
MVC  QSSIB.SSIBLEN,=Y(SSIBSIZE)
MVC  QSSIB.SSIBSSNM,=CL4'JES2'      a quién
MVC  QSTAT.STATEYE,=C'STAT'
MVC  QSTAT.STATLEN,=Y(STATSIZE)
MVI  QSTAT.STATVERL,STATCVRL        versión del área
MVI  QSTAT.STATVERM,STATCVRM
MVI  QSTAT.STATSEL1,STATSJBN        filtro: nombre de job ...
MVC  QSTAT.STATJOBN,PREFIJO          ... con comodines (ZTKD*)
MVI  QSTAT.STATSEL2,STATSJOB        sólo batch
MVI  QSTAT.STATSEL3,STATSPHZ        filtro por fase (STATPHAZ)

```

Antes de armar, **todo el área se limpia con MVCL** (líneas 261–265): un campo que quedó sucio de la vuelta anterior puede cambiar el filtro sin que te des cuenta.

Campo	Valores que usamos
STATTYPE	STATTERS (resumen) · STATVRBO (detalle) · STATMEM (liberar memoria)
STATPHAZ	STAT_EXEC (253, "no terminó de ejecutar") · STAT_POSTEX (254, "terminó")

 Fuente: ZTKMONS.asm rutina INICIA · mapeos IEFSSOBH, IEFJSSIB e IAZSSST.

Bloque	Campo	Para qué
SSOB	SSOBFUNC	Función pedida: 80 (extended status)
SSOB	SSOBSSIB · SSOBINDV	Dirección del SSIB · dirección del área de la función (el STAT)
SSOB	SSOBRETN	Código de retorno de la función (además del R15 de IEFSSREQ)
SSIB	SSIBSSNM	Nombre del subsistema: JES2
STAT	STATTYPE	STATTERS (resumen) · STATVRBO (detalle) · STATMEM (liberar memoria)
STAT	STATSEL1 + STATJOBN	Filtrar por nombre de job; admite comodines (ZTKD*)
STAT	STATSEL2 = STATSJOB	Sólo jobs batch (no STC ni TSU)
STAT	STATSEL3 + STATPHAZ	Filtrar por fase: STAT_EXEC (253, "no terminó") o STAT_POSTEX (254, "terminó")
Terse	STTRPHAZ	Fase detallada del job; ejecutando = STAT_ONMAIN (14) (a confirmar en la demo)
Terse	STTRXIND · STTRMXCC	Cómo terminó (normal,abend,JCL...) · máximo CC
Verbose	STVBXTST · STVBXTET	Hora de inicio y de fin de la ejecución (centésimas desde medianoche) (a confirmar sin APF)

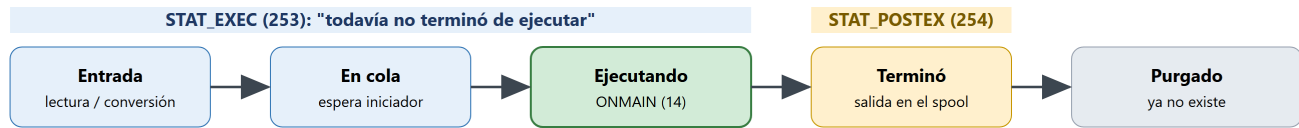
Fuente: ZTKMONS.asm (comentarios de cabecera y LLAMASSI) · mapeo IAZSSST · z/OS MVS Using the Subsystem Interface

Lámina L03-02_campos_clave

3. Las fases de un job

Un job pasa por **entrada** (lectura y conversión) → **en cola** (espera iniciador) → **ejecutando** → **terminó** (la salida queda en el spool) → **purgado**.

- STAT_EXEC** (253) abarca todo lo que **todavía no terminó**. Dentro de eso, el job que está realmente ejecutando tendría **STTRPHAZ = STAT_ONMAIN** (14). **Esto es la pregunta 2 de la demo**. Si resulta falso, ZTKMONS escribe **+DIAG FASE** en vez de **+EVT INI**.
- STAT_POSTEX** (254) es "terminó de ejecutar". De ahí salen los FIN, incluso de jobs que duraron menos que el intervalo.



Por eso, en cada vuelta, ZTKMONS pide EXEC y mira cuáles tienen fase ONMAIN (→ **INI**), y pide POSTEX para ver cuáles terminaron desde la vuelta anterior (→ **FIN**). Así ve también los jobs que duran menos que el intervalo.

✈ Analogía: el tablero del aeropuerto

Un vuelo pasa por "check-in", "embarcando", "en vuelo", "atterizó" y "fuera del tablero". JES tiene un tablero igual para los jobs. El monitor mira el tablero cada pocos segundos y anota los vuelos que **despegaron** (INI) y los que **atterizaron** (FIN).

Los valores 253, 254 y 14 salen de los comentarios de ZTKMONS sobre IAZSSST. Que el ejecutando sea siempre 14 es la **pregunta 2** de la demo.

Fuente: ZTKMONS.asm (cabecera: STAT_EXEC 253, STAT_POSTEX 254, STAT_ONMAIN 14) · mapeo IAZSSST

Lámina L03-03_fases_de_un_job

4. Leer la respuesta: una cadena de bloques

JES deja la respuesta en memoria y la cuelga del área STAT:

Paso	Campo	Qué es
1	STATJOB	Dirección del primer STATJQ (un bloque por job)
2	STJQNEXT	Del STATJQ al siguiente ; 0 = fin
3	STJQOHDR	Desplazamiento desde el STATJQ a su cabecera (STATJQHD)
4	STHDLEN	Largo de todo el bloque de datos del job, no sólo de la cabecera
5	Secciones	Después de la cabecera (STHDSIZE) vienen secciones, cada una con largo (2 bytes) y tipo (1 byte); la terse es STTRTERS
6	STJQVRBO	En una consulta STATVRBO: dirección de la parte verbose del job (secciones de tipo STVBVRBO)

La trampa que ya costó en ZTKJACT (comentario de BUSCATRS, línea 616):

STHDLEN es el largo de **todo** el bloque. Hay que saltar la cabecera con **STHDSIZE** y **recorrer las secciones por su largo y su tipo**, sin suponer que la terse viene primero.

BUSCATRS	LH	R8,QJQ.STJQOHDR	desplazamiento a la cabecera
	AR	R8,R6	R8 = cabecera
	LH	R5,QHD.STHDLEN	largo TOTAL
	LA	R8,STHDSIZE(,R8)	salto la cabecera
	LA	R7,STHDSIZE	
BTSECC	CR	R7,R5	¿me pasé del total?
	BNL	BTNO	
	LH	R0,0(,R8)	largo de esta sección
	CLI	2(R8),STTRTERS	¿es la terse?
	BER	R14	sí → R8 apunta a ella
	AR	R8,R0	no → siguiente sección
	AR	R7,R0	
	B	BTSECC	

Los campos que se leen al final:

Sección	Campo	Para qué
terse	STTRPHAZ	Fase detallada
terse	STTRXIND · STTRMXCC	Cómo terminó (normal,abend,JCL...) · máximo CC
verbose	STVBXTST · STVBXTET	Hora de inicio y de fin de la ejecución, en centésimas desde medianoche. Pregunta 4 de la demo: ¿llegan sin APF?

5. Códigos de retorno y devolver la memoria

Cada pedido deja **tres** códigos, y ZTKMONS los guarda los tres para el +ERR SSI :

Código	Dónde	Qué dice
R15 después de IEFSSREQ	RCSSI	Si el SSI aceptó el pedido (0) o no: subsistema inexistente, inactivo o que no soporta la función (constantes SSRT... de IEFSSOBH)
SSOBRETN	RCSUBSYS	Cómo le fue a la función 80
STATREAS	—	El motivo, si la función no pudo

Devolver la memoria: la respuesta ocupa memoria que JES obtuvo para vos. Después de leerla hay que hacer **otro** pedido con STATTYPE = STATMEM (rutina LIBERA, líneas 300–307). Si no, en un monitor que pregunta cada 5 s toda la noche, la memoria se acumula vuelta tras vuelta.

6. Una vuelta de ZTKMONS

#	Pedido	Qué obtiene	Qué escribe
0	Al arrancar: POSTEX sin escribir	Los jobs que ya habían terminado	+BASE YA TERMINADOS N=...
1	STATTERS , fase EXEC	Tabla NUEVA de jobs ejecutando	—
2	Comparar NUEVA con VIEJA	Jobid nuevo = arrancó	+EVT INI ...
3	STATVRBO , fase POSTEX	Terminados, con horas y CC	+EVT FIN ... de los que no estaban en REPORT
4	STATMEM	Devolver la memoria	—
5	NUEVA pasa a ser VIEJA	—	Cada 12 vueltas, +PONG

Límites conocidos de v0:

- Tablas fijas: 200 ejecutando y 2.000 reportados.
- T/TI llevan la fecha de hoy, así que un job que cruza la medianoche sale con la fecha del día siguiente.

Una vuelta de ZTKMONS, paso a paso

L03 · Hablarle a JES por SSI 80

#	Pedido	Qué obtiene	Qué escribe
0	Al arrancar: POSTEX sin escribir	Los jobs que ya habían terminado	+BASE YA TERMINADOS N=... (no se reportan)
1	STATTERS , fase EXEC, prefijo del PARM	Tabla NUEVA de jobs ejecutando	—
2	Comparar NUEVA con VIEJA	Jobid en NUEVA y no en VIEJA = arrancó	+EVT INI ...
3	STATVRBO , fase POSTEX	Jobs terminados, con horas STVBXTST/XTET y CC	+EVT FIN ... de los que no estaban en REPORT
4	STATMEM	Devolver la memoria de las respuestas	—
5	NUEVA pasa a ser VIEJA	—	Cada 12 vueltas: +PONG (latido)

Analogía: dos fotos y el juego de las diferencias

Cada vuelta saca una **foto** de quién está ejecutando (NUEVA) y la compara con la foto anterior (VIEJA): los que aparecen son INI. Aparte lleva una **libreta** (REPORT) con los FIN que ya anotó, para no repetirlos.

Límites conocidos de v0: tablas fijas (200 ejecutando, 2.000 reportados) y fecha de hoy en T/TI (un job que cruza la medianoche sale con fecha del día siguiente).

Fuente: ZTKMONS.asm (cabecera "COMO SABE QUE PASO" y "LIMITES CONOCIDOS")

1. ¿Para qué sirven por separado el R15 de `IEFSSREQ` y el `SSOBRETN`? Poné un ejemplo donde el primero dé 0 y el segundo no.
2. ¿Qué pasaría si un monitor que pregunta cada 5 s nunca hiciera el pedido `STATMEM`?
3. ¿Por qué `ZTKMONS` pide `POSTEX` una vez al arrancar **sin escribir** nada?
4. Un job dura 2 s y el intervalo es de 5 s. ¿Sale su `+EVT INI`? ¿Y su `+EVT FIN`? ¿Por qué?
5. ¿Cuál es "la trampa de `ZTKJACT`" y qué pasaría si se ignorara?

Mis respuestas

Un programa que hace **una sola** consulta STATTERS, fase EXEC, con el prefijo del PARM. Escribe una línea por job (`jobname jobid dueño clase fase`) y termina, liberando la memoria con STATMEM.

1. Copiá INICIA, LLAMASSI, LIBERA y BUSCATRS de ZTKMONS.
2. Corrélo con `PARM='*'` mientras tenés otro job tuyo ejecutando, por ejemplo tu ZTKBUCLE de P02.
3. Traé el SYSPRINT y decime si **tu** ZTKBUCLE salió con fase 14 (ONMAIN).
Esa respuesta **adelanta la pregunta 2 de la demo**.

Relacionado: [L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB](#) ·
[Glosario](#) · siguiente: [L04 - ZTKMONS v0 en DESA - ensamblar y la demo en modo spool](#).

 Fuente: `vivo\host\ZTKMONS.asm` (INICIA, LLAMASSI, LIBERA, ERRSSI, BUSCATRS, BUSCAVRB) · *z/OS MVS Using the Subsystem Interface* · mapeo IAZSSST.

L04 — ZTKMONS v0 en DESA: ensamblar y la demo en modo spool

Objetivo: convertir el fuente de ZTKMONS en un programa que corre, leer sus errores, correr la demo con los jobs de prueba y **contestar con datos** las cinco preguntas abiertas. Es la **primera validación real** del monitor.

1. De fuente a programa

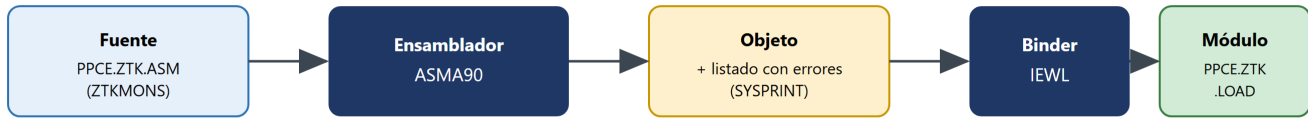
Paso	Programa	Entra	Sale
Ensamblar	ASMA90 (HLASM)	PPCE.ZTK.ASM(ZTKMONS)	Módulo objeto + listado con errores (SYSPRINT)
Enlazar	IEWL (binder)	El objeto	Módulo de carga PPCE.ZTK.LOAD(ZTKMONS)

Así lo hace `vivo\host\ZTKMONA.jcl`, o tu procedimiento habitual de ZTK si preferís:

```
//ASM      EXEC PGM=ASMA90,PARM='OBJECT,NODECK,XREF(SHORT)'  
//SYSLIB   DD DSN=SYS1.MACLIB,DISP=SHR          macros (STIMER, OPEN, I  
//         DD DSN=SYS1.MODGEN,DISP=SHR          mapeos de sistema (CVT,  
//SYSLIN   DD DSN=&&OBJ,DISP=(NEW,PASS),...      el objeto, temporal  
//SYSIN    DD DSN=PPCE.ZTK.ASM(ZTKMONS),DISP=SHR  
//LKED     EXEC PGM=IEWL,COND=(4,LT,ASM),        no enlaza si ASMA90 dio  
//         PARM='LIST,XREF,AMODE=31,RMODE=24'  
//SYSLMOD  DD DSN=PPCE.ZTK.LOAD(ZTKMONS),DISP=SHR
```

- **SYSLIB con MACLIB y MODGEN:** si falta una de las dos, el ensamblador no encuentra una macro o un mapeo, y los errores se encadenan.
- **COND=(4,LT,ASM) :** el binder no corre si el ensamblado dio más de 4. Así no se reemplaza un módulo bueno por uno roto.
- **AMODE=31,RMODE=24** en el binder: lo explicamos en L01.

 Fuente: `vivo\host\ZTKMONA.jcl` · *HLASM Programmer's Guide · z/OS MVS Program Management: User's Guide and Reference* (binder).



El job de ensamblado es vivo\host\ZTKMONA.jc1 (o tu procedimiento habitual de ZTK). Se corre con ZTKMONR.jc1.

RC de ASMA90	Qué significa	Qué hacés
0	Sin mensajes	Seguir
4	Advertencias	Leerlas; en general se sigue
8	Errores: probablemente el programa no hace lo que pensás	Corregir
12 / 16	Errores graves / terminales	Corregir

Analogía: la receta, la cocina y el plato

La fuente es la **receta**. El ensamblador la **traduce** a ingredientes preparados (el objeto) y te devuelve una lista de lo que no entendió. El binder **arma el plato** listo para servir (el módulo de carga), que es lo que el job ejecuta.

Fuente: HLASM Programmer Guide (códigos de retorno del ensamblador) · vivo\host\LEEME.md

Lámina L04-01_de_fuente_a_programa

2. Leer el listado de ASMA90

RC de ASMA90	Qué significa	Qué hacés
0	Sin mensajes	Seguir
4	Advertencias	Leerlas; en general se sigue
8	Errores: el programa probablemente no hace lo que pensás	Corregir
12 / 16	Errores graves o terminales	Corregir

Cómo encontrar el error:

1. Al final del listado hay un **resumen de diagnósticos** con el número de sentencia de cada mensaje.
2. Cada mensaje tiene la forma `ASMAnnnX`, donde la **letra final** es la gravedad (I, W, E, S...).
3. Los más comunes en un primer ensamblado son **símbolo no definido** y **falta un USING activo** para un operando. Este último es la promesa de

L01 que no se cumplió.

4. **Corregí el primero primero.** Un error al principio suele generar otros diez en cascada.

Los números exactos de cada mensaje los vas a ver en tu listado: los anotamos en la práctica, no de memoria.

3. Correr el monitor: ZTKMONR

```
//MONITOR EXEC PGM=ZTKMONS,PARM='ZTKD* 5 20',TIME=30
//STEPLIB DD DSN=PPCE.ZTK.LOAD,DISP=SHR
//SYSPRINT DD SYSOUT=*,DCB=(RECFM=FB,LRECL=133,BLKSIZE=13300)
```

Detalle	Por qué
PARM= 'ZTKD* 5 20'	Mira los jobs ZTKD*, cada 5 s, durante 20 min (240 vueltas)
TIME=30	Tope de 30 min de CPU: si algo sale mal, no gira para siempre
El job se llama ZTKMONR	No puede empezar con ZTKD: se vería a sí mismo

4. La demo, paso a paso

Paso	Qué hacés	Qué tiene que pasar
1	Subir ZTKMONS.asm y correr ZTKMONA	RC ≤ 4 y PPCE.ZTK.LOAD(ZTKMONS) creado. Nunca se ensambló: los errores aparecen acá
2	Someter ZTKMONR	Queda ACTIVE
3	Someter los jobs de prueba de vivo\demo_desa\ (tabla siguiente)	Corren normal: el monitor sólo mira
4	Al terminar ZTKMONR: python vivo\spool_a_portal.py <JOBID>	Informe en vivo\validaciones\<JOBID>.md y alertas en la pestaña «En vivo»

Job de prueba	Qué hace	Qué tiene que decir el monitor
---------------	----------	--------------------------------

ZTKDOK ×12, SEG entre 20 y 40	BPXBATCH sleep	INI y FIN COMP=NRM CC=0000 ; el portal aprende la duración
ZTKDRC8	IDCAMS borra un DSN que no existe	FIN ... CC=0008 → FALLA
ZTKDABN	EXEC PGM=ZTKNOEXI	FIN ... COMP=ABN CC=S806 → FALLA
ZTKDJCL	DISP=(NUEVO,...) inválido	¿Aparece en POSTEX? (pregunta 3)
ZTKDNUE	sleep 30 , nombre nunca visto	Arranque en frío (umbral 300 s), sin falsa alarma
ZTKDOK SEG=600	El mismo job, durmiendo 10 min	COLGADO al pasar el umbral aprendido
Ráfaga: 50 × ZTKDOK	Muchos seguidos	Ningún evento perdido

Los jobs de prueba no leen ni escriben datos. Antes de someterlos, ajustá la tarjeta JOB (cuenta, CLASS, MSGCLASS) a lo que use DESA.

La demo en modo spool, paso a paso

L04 - ZTKMONS v0 en DESA

Paso	Qué hacés	Qué tiene que pasar
1	Subir ZTKMONS.asm a PPCE.ZTK.ASM(ZTKMONS) y correr ZTKMONA.jc1	ASMA90 con RC ≤ 4 y PPCE.ZTK.LOAD(ZTKMONS) creado. Nunca se ensambló: los errores aparecen acá
2	Someter ZTKMONR.jc1 (PARM 'ZTKD* 5 20' : cada 5 s durante 20 min)	Queda ACTIVE
3	Mientras corre, someter los jobs de prueba: 12 × ZTKDOK (SEG 20–40), ZTKDRC8, ZTKDABN, ZTKDJCL, ZTKDNUE y al final ZTKDOK con SEG=600	Los jobs corren normal: el monitor sólo mira
4	Cuando ZTKMONR termine: python vivo\spool_a_portal.py <JOBID>	Informe en vivo\validaciones\<JOBID>.md y alertas en la pestaña «En vivo»

Job de prueba	Qué simula	Qué debería decir el monitor
ZTKDOK	Job normal de 20–40 s	INI y FIN ... COMP=NRM CC=0000 ; enseña el umbral
ZTKDRC8	Termina con RC 8	FIN ... CC=0008 → alerta FALLA
ZTKDABN	Abend	FIN ... COMP=ABN CC=S...
ZTKDJCL	Error de JCL	¿Aparece en POSTEX? (pregunta 3)
ZTKDNUE	Job nunca visto	Umbral de arranque en frío
ZTKDOK SEG=600	Job que "se cuelga" (10 min)	Alerta COLGADO

Fuente: vivo\host\LEEME.md

5. Qué valida spool_a_portal.py

Chequeo	Contra qué se compara
El SYSPRINT tiene sólo líneas del protocolo, sin +ERR del SSI	El formato de PROTOCOLO.md
El contador de eventos cuadra	+FIN ZTKMON ... EVENTOS=n vs. las +EVT contadas
La hora del monitor es correcta	El IEF403I ... STARTED del JESMSG LG de cada job
El CC de cada FIN es el real	El retcode que da JES para ese jobid
No se perdió ningún job	Todos los ZTKD* sometidos después del monitor
Qué fases reportó JES2	Las líneas +DIAG

Qué valida spool_a_portal.py (✓ / X)

L04 · ZTKMONS v0 en DESA

Chequeo	Contra qué se compara
El SYSPRINT tiene sólo líneas del protocolo, sin +ERR del SSI	El formato de PROTOCOLO.md
El contador de eventos cuadra	+FIN ZTKMON ... EVENTOS=n vs. las líneas +EVT contadas
La hora del monitor es correcta	El IEF403I ... STARTED del JESMSG LG de cada job
El CC de cada +EVT FIN es el real	El retcode que da JES para ese jobid
No se perdió ningún job	Todos los ZTKD* sometidos después del monitor
Qué fases reportó JES2	Las líneas +DIAG (confirma si el ejecutando es ONMAIN)

Analogía: el auditor con el extracto del banco

El monitor es tu **libreta de gastos**; JES es el **extracto oficial**. El script es el auditor que compara línea por línea y marca con ✓ lo que coincide y con X lo que no.

Fuente: vivo\host\LEEME.md (sección "Qué valida el paso 4")

Lámina L04-03_que_valida

6. Las cinco preguntas y dónde se lee cada respuesta

#	Pregunta	Dónde mirar
1	¿ TIME DEC,...,DATETYPE=YYYYMMDD deja la fecha en los bytes 8–11?	La fecha del +HOLA: ¿es la de hoy?
2	¿El que ejecuta tiene STTRPHAZ = 14 ?	Si hay +DIAG FASE y ningún INI, no

3	¿Un error de JCL aparece en POSTEX?	¿Hay +EVT FIN de ZTKDJCL?
4	¿STATVRBO trae STVBXTST / STVBXTET sin APF?	¿Los TI= de los FIN tienen hora?
5	¿Cuánta CPU gasta cada vuelta?	CPU del paso de ZTKMONR (IEF032I) ÷ 240, comparada con P02

Cada respuesta va a 20 - Practicas con su evidencia (JOBID y línea del SYSPRINT). Si alguna sale distinta de lo esperado, se corrige ZTKMONS **y la lámina o la lección que lo suponía.**

7. Preguntas de comprobación

1. ¿Por qué el paso del binder tiene COND=(4,LT,ASM) ? ¿Qué pasaría sin él si el ensamblado diera 8?
2. ¿Por qué ZTKMONR no puede llamarse ZTKDMON?
3. Si el listado tiene 40 errores, ¿por dónde empezás y por qué?
4. ¿Qué te dice que aparezcan +DIAG FASE y ningún +EVT INI ?
5. ¿Cómo calculás la CPU que agrega SSI 80, usando el número de P02?

Seguí la sección 4 y traé:

- Relacionado: [L03 - Hablarle a JES por SSI 80](#) · siguiente: [L05 - Diseño del protocolo ZTKMON v1](#).

Monitor z · cuaderno de estudio

L05 — Diseño del protocolo ZTKMON v1

Objetivo: entender **por qué** el protocolo es como es, para poder cambiarlo sin romperlo. Un protocolo es un contrato: si el host lo cumple, la PC funciona sin cambios, y viceversa.

1. Las decisiones de base

Decisión	Alternativa descartada	Por qué
Texto , una línea por mensaje	Registros binarios	Se lee a ojo en el SYSPRINT (modo spool) y en un volcado de red; un error se ve sin herramientas
EBCDIC , la traducción la hace la PC	Que el host traduzca a ASCII	El host queda simple; el puente decodifica con <code>cp037</code>
Sólo mayúsculas, dígitos, blanco y <code>+ = : - . /</code>	Cualquier carácter	Son iguales en las páginas 037 y 1047 : no importa cuál use el host
Fin de línea X'15' (NL)	CR/LF	Es el fin de línea de EBCDIC, como el resto de ZTK
Máx. 200 bytes; campos con un blanco; opcionales <code>CLAVE=valor</code>	Posiciones fijas	Se pueden agregar campos sin romper al que lee
<code>V=1</code> en el <code>+HOLA</code>	Sin versión	Si cambia el contrato, cada lado sabe con quién habla
Horas locales del host, <code>AAAA-MM-DD-HH:MM:SS</code>	Hora UTC	Coincide con la bitácora y con el JESMSGLG

2. Mensajes del host a la PC

Línea	Cuándo	Campos clave
<code>+HOLA ZTKMON</code> <code>V=1 SYS= JES=</code> <code>T=</code>	Al aceptar la conexión	Versión, sistema, JES

+EVT INI job jobid OW= CL= T=	El job empezó a ejecutar	Dueño, clase, hora
+EVT FIN job jobid ... T= TI= COMP= CC=	Terminó	TI = inicio, sin él no se aprende la duración . COMP : NRM, CC, ABN, JCL, CAN, FLU, SEC, SYS, UNK. CC : 0008, S806, U0050...
+EVT FASE / +EVT PURGA	Cambio de fase / purga (opcionales)	Q= fase
+ACT ... y +FINACT N=	Respuesta a ACT	Uno por job ejecutando; cierra con la cantidad
+FINHIST N=	Cierra la respuesta a HIST	Cantidad repetida
+PONG T=	Respuesta a PING	Latido
+ERR texto	Pedido inválido o error	Nunca se corta la conexión por un pedido malo
+DIAG FASE ...	Diagnóstico	El portal lo ignora; sirve para validar

Línea	Cuándo	Campos clave
+HOLA ZTKMON V=1 SYS= JES= T=	Al aceptar la conexión	Versión del protocolo, sistema, JES
+EVT INI job jobid OW= CL= T=	El job empezó a ejecutar	Dueño, clase, hora
+EVT FIN job jobid ... T= TI= COMP= CC=	Terminó	TI = inicio (sin él no se aprende la duración) · COMP : NRM, CC, ABN, JCL, CAN, FLU, SEC, SYS, UNK · cc : 0008, S806, U0050...
+EVT FASE / +EVT PURGA	Cambio de fase / purga (opcionales)	Q= fase
+ACT ... y +FINACT N=	Respuesta a ACT	Uno por job ejecutando; cierra con la cantidad
+FINHIST N=	Cierra la respuesta a HIST	Cantidad de FIN repetidos
+PONG T=	Respuesta a PING	Latido
+ERR texto	Pedido inválido o error	Nunca se corta la conexión por un pedido malo
+DIAG FASE ...	Diagnóstico de fases	El portal lo ignora; sirve para validar

Regla de formato	Valor
Código y fin de línea	EBCDIC (037/1047), cada línea termina en X'15' (NL)
Caracteres	Mayúsculas, dígitos, blanco y + = : - . / (iguales en 037 y 1047)
Tamaño y campos	Máx. 200 bytes; separados por un blanco; opcionales como CLAVE=valor
Hora	Local del host: AAAA-MM-DD-HH:MM:SS

Fuente: plataforma_cierres\vivo\PROTOCOLO.md

Lámina L05-01_mensajes_host_a_pc

3. Pedidos de la PC al host

Pedido	Qué pide	Respuesta
SUB	Empezar a recibir +EVT	Los eventos, a medida que pasan
ACT	Jobs ejecutando ahora (reconciliar al conectar)	+ACT × n y +FINACT N=
JOB jobid	Estado de un job puntual	+EVT INI o +EVT FIN
HIST ts	Repetir los FIN desde ts	+EVT FIN × n y +FINHIST N=
PING	Latido	+PONG
ADIOS	Cierre ordenado	—

No existe ningún pedido que cambie algo en el host: ni cancelar, ni reejecutar, ni liberar, ni tocar TWS. Esto es una **decisión de seguridad**, no un faltante. Un monitor que puede actuar es un riesgo para producción, y quien lo opera no tiene por qué tener ese poder.

Pedidos de la PC al host			L05 · Diseño del protocolo ZTKMON v1
Pedido	Qué pide	Respuesta	
SUB	Empezar a recibir +EVT (suscripción)	Los eventos, a medida que pasan	
ACT	Lista de jobs en ejecución (reconciliar al conectar)	+ACT × n y +FINACT N=	
JOB jobid	Estado de un job puntual	+EVT INI o +EVT FIN	
HIST ts	Repetir los FIN de los jobs que terminaron desde ts	+EVT FIN × n y +FINHIST N=	
PING	Latido	+PONG	
ADIOS	Cierre ordenado	—	

No existe ningún pedido que cambie algo en el host (cancelar, reejecutar, liberar, tocar TWS). Un pedido desconocido se responde con +ERR y se ignora.

 **Analogía: el walkie-talkie con código de radio**
Las dos partes usan frases fijas y cortas ("¿me copiás?" = PING, "copiado" = PONG). Si alguien dice algo fuera del código, se contesta "no entendí" (+ERR) y la charla sigue. Y por esta radio **sólo se informa**: no se dan órdenes.

Fuente: plataforma_cierres\vivo\PROTOCOLO.md

Lámina L05-02_mensajes_pc_a_host

4. Conexión, latido y reconexión

1. Al conectar, la PC manda SUB y ACT, más HIST <última hora vista> si es una reconexión.
2. Los +ACT se cargan como jobs corriendo. Los que el portal tenía corriendo y **no** vinieron se consultan uno por uno con JOB jobid.
3. PING cada 30 s. Sin +PONG en 90 s, la PC reconecta con esperas crecientes: 5, 10, 20, 40 y 60 s.
4. Cada línea se traduce a JSON y va a POST /api/vivo/observaciones.
5. El portal marca **COLECTOR caído** a los 180 s sin datos ni latido.

Las reglas que salieron de errores reales (prueba de punta a punta, 2026-09-30):

Regla	Lo que pasaba antes
-------	---------------------

HIST <última hora> sale del portal , no del puente	Un puente reiniciado no sabía desde cuándo pedir: se recuperaban 55 de 63 . Ahora, 63 de 63
COLECTOR caído a los 180 s, más que el latido	Con un plazo menor que el latido (30 s), saltaba en falso
El FIN trae TI=	Un FIN sin su INI (el puente conectó tarde) quedaba con duración 0

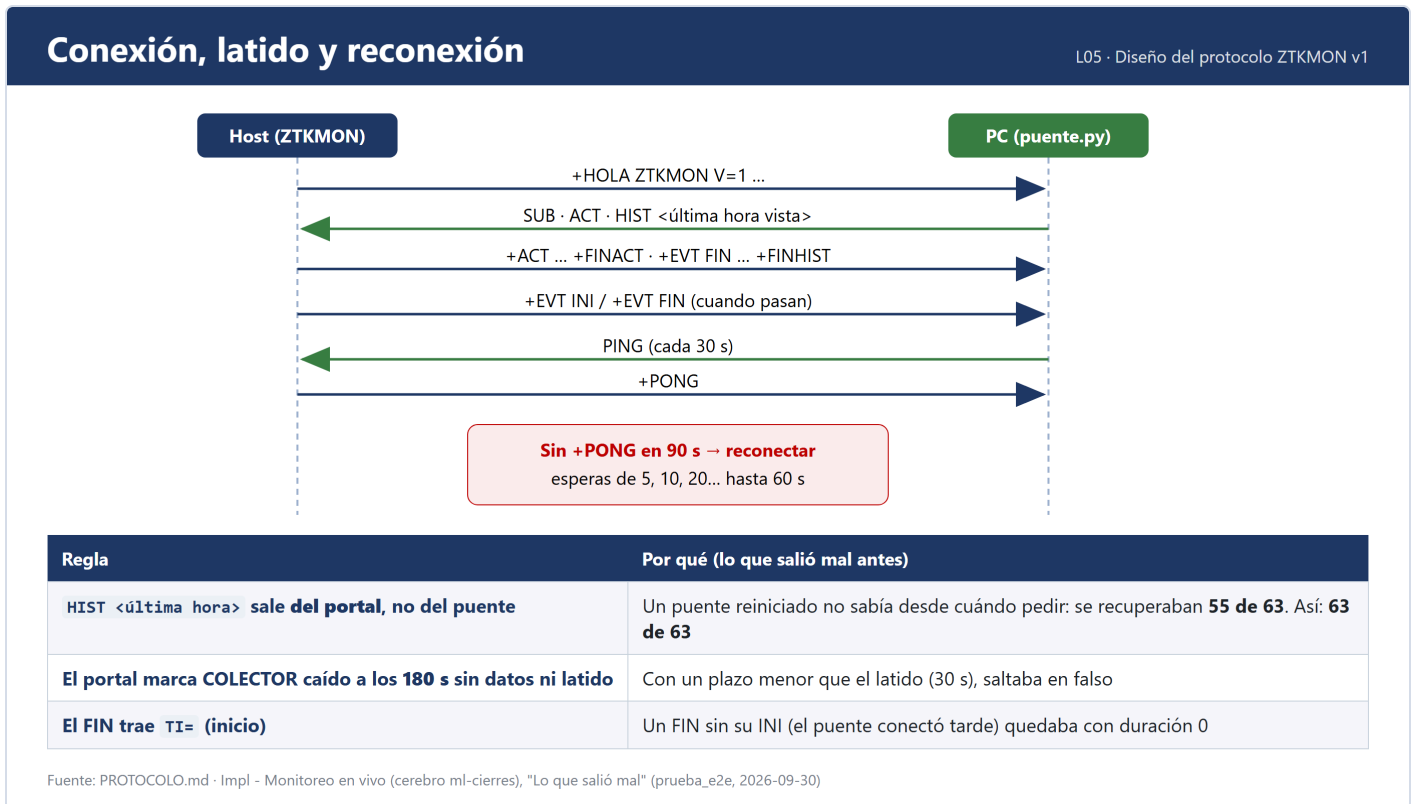


Lámina L05-03_conexion_y_reconexion

5. Tres propiedades que hay que cuidar

- **Idempotencia:** después de un HIST, el portal puede recibir **dos veces** el mismo FIN. Tiene que reconocerlo por su jobId y no contarlos dos veces.
→ Lo verificás en la práctica.
- **Tolerancia:** un pedido desconocido no rompe nada (+ERR) y un campo opcional nuevo no rompe al lector viejo.
- **Reconciliación:** nunca confiar en que no se perdió nada. Al conectar siempre se pide ACT, y al reconectar, HIST.

1. ¿Por qué el protocolo usa sólo los caracteres `+ = : - . /` además de letras y dígitos?
2. Si mañana se agrega un campo `CPU=` al `+EVT FIN`, ¿qué tiene que pasar para que el puente viejo no se rompa?
3. ¿Qué pasaría si el plazo de `COLECTOR` fuera de 20 s con un latido de 30 s?
4. Explicá con tus palabras por qué `HIST` tiene que salir del portal y no del puente.
5. ¿Qué riesgo concreto evita no tener un comando `CANCEL` ?

Mis respuestas

1. Corrél `plataforma_cierres\vivo\iniciar_demo.bat` 4: simulador, puente y portal, 4 veces más rápido. Mirá la pestaña «En vivo».

2. Corré `python plataforma_cierres\vivo\prueba_e2e.py`. Tiene que dar **11/11**.
3. En `plataforma_cierres\app\vivo.py`, busca la función `_evento` y contestá: **¿qué pasa si llega dos veces el mismo FIN?** ¿Se cuenta dos veces para el umbral?
4. Leé `sim_ztkmon.py`: es un ZTKMON de mentira que cumple el protocolo. Es la referencia de cómo tiene que comportarse el de verdad.

Relacionado: [L04 - ZTKMONS v0 en DESA - ensamblar y la demo en modo spool](#) · siguiente: [L06 - Sockets en z-OS](#).

 Fuente: `plataforma_cierres\vivo\PROTOCOLO.md` · `vivo\puente.py` · nota
Impl - Monitoreo en vivo del cerebro ml-cierres ("Lo que salió mal").

L06 — Sockets en z/OS

Objetivo: que ZTKMON deje de escribir en SYSPRINT y **hable en vivo** con el puente de la PC por TCP. Al final sabés la secuencia de un servidor, por qué no hay que bloquearse sin plazo y cómo se arman las líneas que llegan partidas.

⚠ **Requiere de sistemas:** un **puerto TCP** reservado en DESA y una regla de firewall PC ↔ host. Sin eso, la teoría se estudia igual y la práctica se hace cuando estén.

1. Quién escucha y quién llama

En nuestro diseño **el host es el servidor**: ZTKMON escucha en un puerto. **La PC es el cliente**: el puente se conecta.

Decisión	Por qué
El host escucha	El puerto se reserva una vez en el host, y la PC puede reconectarse cuando quiera
Un cliente a la vez	Suficiente para la demo; simplifica el programa
El host rechaza IPs que no estén en su lista	Que no se conecte cualquiera de la red

2. La secuencia de un servidor TCP

z/OS ofrece una API de sockets para Assembler: la **macro EZASMI**. También existe la interfaz por `CALL` (EZASOKET), pero acá usamos la macro.

#	Llamada	Qué hace	Analogía 📞
1	INITAPI	Conecta el programa con TCP/IP	Contratar la línea
2	SOCKET	Crea un socket de tipo <i>stream</i> (TCP)	Comprar el aparato
3	BIND	Lo asocia al puerto	Asignarle el número
4	LISTEN	Lo pone a escuchar	Encender el timbre

5	ACCEPT	Atiende una conexión y devuelve otro socket para esa charla	Atender la llamada
6	READ / WRITE (o RECV / SEND)	Recibe y manda bytes	Hablar y escuchar
7	CLOSE	Cierra la charla (o el socket que escucha)	Colgar
8	TERMAPI	Se desconecta de TCP/IP	Dar de baja la línea

Cada llamada devuelve un código de retorno y un `ERRNO`. El `ERRNO` dice qué falló (puerto en uso, conexión cortada...) y se escribe siempre en el log del monitor.

 Fuente: z/OS Communications Server: IP Sockets Application Programming Interface Guide and Reference (macro EZASMI). Los nombres exactos de los parámetros se toman del manual al escribir P06.

Un servidor TCP: la secuencia de llamadas

L06 · Sockets en z/OS

#	Llamada (EZASMI)	Qué hace	Analogía del teléfono 📞
1	INITAPI	Conecta el programa con TCP/IP	Contratar la línea
2	SOCKET	Crea el socket (stream = TCP)	Comprar el aparato
3	BIND	Lo asocia a un puerto	Asignarle un número
4	LISTEN	Lo pone a escuchar conexiones	Encender el timbre
5	ACCEPT	Atiende una conexión y devuelve un socket nuevo para esa charla	Atender la llamada
6	READ / WRITE (o RECV / SEND)	Recibe y manda bytes	Hablar y escuchar
7	CLOSE	Cierra la charla (o el socket que escucha)	Colgar
8	TERMAPI	Se desconecta de TCP/IP	Dar de baja la línea

En nuestro diseño el host es el servidor (escucha en un puerto que asigna sistemas) y la PC es el cliente (se conecta). Además, el host rechaza IPs que no estén en su lista.

Fuente: z/OS Communications Server: IP Sockets Application Programming Interface Guide and Reference (EZASMI) · PROTOCOLO.md

Lámina L06-01_servidor_tcp

3. Bloqueo y plazos: la regla más importante

Por defecto, `ACCEPT` y `READ` son **bloqueantes**: la tarea queda esperando hasta que llegue algo.

- Si el puente no manda nada, el monitor **deja de vigilar** y no atiende un `P ZTKMON`.
- **Antecedente propio:** en ZTKAVISA (que es cliente), un `CONNECT` sin plazo tardó **77 s** en devolver el error.

Las dos salidas:

Opción	Cómo	Cuándo conviene
<code>SELECT</code> con timeout	Preguntar "¿hay algo para leer en estos sockets?" con un plazo; si no hay nada, se vuelve al bucle	La más simple de razonar
Modo asíncrono con ECB	La API postea un ECB cuando la operación termina; el ECB se suma a la <code>ECBLIST</code> de L02	Para que socket, timer y operador despierten al mismo <code>WAIT</code>

A confirmar en la práctica: cuál de las dos se usa y con qué parámetros exactos de EZASMI.

Bloqueo, plazos y por qué importa

L06 · Sockets en z/OS

❌ Llamada bloqueante sin plazo

`READ` espera **para siempre** si la PC no manda nada. Mientras tanto, el monitor **no vigila** y no atiende un `P ZTKMON`.

Antecedente: en ZTKAVISA (cliente) un `CONNECT` sin plazo tardó **77 s** en devolver el error.

✅ Esperar con plazo

Antes de leer, preguntar "¿hay algo para leer?" con `SELECT` y un **timeout**, o usar la forma asíncrona del API con un **ECB**. Así el socket se suma a la lista de cosas que despiertan al bucle (L02).

La variante exacta se elige y se prueba en la lección.

Tema	Qué tener en cuenta
Puerto	Lo reserva sistemas en el perfil de TCP/IP (sentencia <code>PORT</code>) para el STC
EBCDIC / ASCII	El host manda EBCDIC; la traducción a texto la hace el puente en la PC
Un cliente a la vez	Suficiente para la demo; el host rechaza IPs fuera de su lista
Mensajes partidos	TCP es un flujo : una línea puede llegar en dos pedazos. Se arma hasta encontrar <code>X'15'</code>

■ Analogía: esperar a alguien en la puerta

Bloqueante sin plazo es quedarse parado en la puerta hasta que llegue, aunque se queme la cocina. Con plazo es mirar por la ventana cada tanto y seguir con lo tuyo mientras tanto.

Fuente: EZASMI Guide and Reference (SELECT, modo asíncrono) · IDEA-005 (ZTKAVISA, CONNECT de 77 s) · PROTOCOLO.md

Lámina L06-02_bloqueo_y_plazos

4. TCP es un flujo, no mensajes

TCP entrega **bytes**, no líneas:

- Un `READ` puede traer **media línea**, una línea y media o tres líneas juntas.
- **Regla:** acumular en un buffer y cortar **en cada X'15'**. Lo que queda después del último X'15' espera al próximo `READ`.
- Al escribir, mandar la línea completa con su X'15'. Si `WRITE` dice que mandó menos bytes que los pedidos, mandar el resto.
- Del lado PC esto ya está resuelto: `punte.py` lee línea por línea y decodifica con `cp037`.

5. Qué hay que pedir y qué se configura

Tema	Detalle
Puerto	Lo reserva sistemas en el perfil de TCP/IP (sentencia <code>PORT</code>), asociado al STC
Firewall	Regla desde la IP de la PC hacia el puerto del host
Lista de IPs permitidas	La lleva el propio ZTKMON (PARM o miembro de configuración)
Translation	El host manda EBCDIC; la PC traduce. El host no convierte nada

6. Preguntas de comprobación

1. ¿Por qué `ACCEPT` devuelve un socket **nuevo** en lugar de usar el que escucha?
2. Llega un `READ` con `+EVT INI ZTKDOK JOB01` y nada más. ¿Qué hace el programa con esos bytes?
3. ¿Qué le pasa al monitor si hace `READ` sin plazo y el puente de la PC se cuelga?
4. ¿Por qué conviene que el socket también despierte el `WAIT ECBLIST`?
5. ¿Para qué sirve que el host tenga su propia lista de IPs permitidas, si ya hay un firewall?

 Fuente: *IP Sockets API Guide and Reference* (EZASMI) · PROTOCOLO.md (transporte) ·
puente.py · IDEA-005 (ZTKAVISA, CONNECT de 77 s).

L07 — Ser un STC de verdad

Objetivo:

- que el monitor arranque con `S ZTKMON`;
- que atienda órdenes del operador (`F ZTKMON, ESTADO`);
- que pare **ordenado** con `P ZTKMON`;
- que, si algo falla, deje dicho qué pasó.

 **Requiere de sistemas:** el miembro de la PROC en la PROCLIB y el perfil en la clase STARTED. La parte de órdenes del operador **se puede practicar antes** con un job batch (ver la práctica).

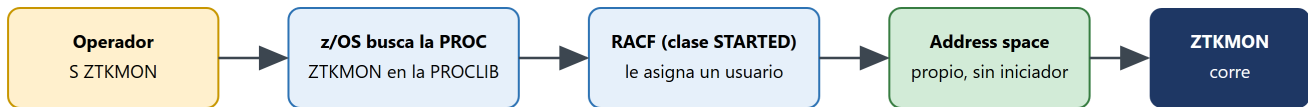
1. Qué cambia de job a STC

	Job batch largo	STC
Arranca con	<code>SUBMIT</code>	<code>S ZTKMON</code> (o el arranque del sistema)
JCL	El miembro que sometés	Una PROC en la PROCLIB
Iniciador	Ocupa uno todo el tiempo	No usa iniciador batch: tiene su propio address space
Usuario	El tuyo	El que asigna RACF por la clase STARTED
Se para con	Cancelar (o que termine solo)	<code>P ZTKMON</code> : ordenado

El programa es casi el mismo. Lo que cambia es cómo se arranca y que **tiene que atender al operador.**

2. Cómo arranca

1. El operador escribe `S ZTKMON`.
2. z/OS busca la PROC `ZTKMON` en la PROCLIB.
3. RACF le asigna el usuario según el perfil STARTED.
4. Se crea el address space y se ejecuta el programa.



Lo que hay que pedirle a sistemas: el miembro en la PROCLIB, el perfil en la clase STARTED, y (si usa ENF 70) que la librería sea APF.

Analogía: el empleado de planta

Para que un empleado de planta empiece a trabajar hace falta su **contrato** (la PROC en la PROCLIB), su **credencial** (el usuario de la clase STARTED) y su **oficina** (el address space). Después llega solo cada mañana cuando lo llaman (S ZTKMON).

Fuente: z/OS MVS System Commands (START) · z/OS Security Server RACF (clase STARTED) · criterio propio para el diagrama

Lámina L07-01_como_arranca_un_stc

3. Recibir órdenes: EXTRACT, COM, CIB y QEDIT

#	Paso	Detalle
1	EXTRACT answer, FIELDS=COMM	Deja en <code>answer</code> la dirección del área COM (mapeo IEZCOM)
2	COMECBPT	Dirección del ECB que el sistema postea cuando llega un comando → va a la <code>ECBLIST</code> del bucle (L02)
3	COMCIBPT	Dirección del primer CIB (mapeo IEZCIB): el comando en sí
4	CIBVERB	Qué fue: START, MODIFY (F) o STOP (P)
5	CIBDATLN / CIBDATA	Largo y texto del MODIFY, por ejemplo ESTADO
6	QEDIT ORIGIN=..., BLOCK=...	Liberar el CIB atendido; si no, se acumulan
7	QEDIT ORIGIN=..., CIBCTR=n	Permitir hasta <i>n</i> MODIFY encolados

Detalles a confirmar en la práctica (se toman de los mapeos IEZCOM/IEZCIB de tu SYS1.MACLIB):

- Los valores exactos de `CIBVERB`.
- Si al arrancar con parámetros queda un **CIB de START** encolado. Si queda, hay que leerlo (trae los parámetros) y liberarlo con QEDIT antes de entrar

al bucle.

Lo que haría ZTKMON con cada orden (propuesta):

Orden	Acción
F ZTKMON, ESTADO	WTO con vueltas, eventos, clientes conectados y CPU
P ZTKMON	Cerrar el socket, mandar +FIN, cerrar SYSPRINT y terminar con RC 0
Cualquier otro texto	WTO "orden desconocida", y nada más

 Fuente: z/OS MVS Programming: Assembler Services Guide, "Communicating with the system operator" (EXTRACT, QEDIT, CIB).

Recibir F y P del operador: EXTRACT, CIB y QEDIT

L07 · Ser un STC de verdad

#	Paso	Detalle
1	EXTRACT ... FIELDS=COMM	Devuelve la dirección del área COM (mapeo IEZCOM)
2	En el COM: COMECBPT	Dirección del ECB que el sistema postea cuando llega un comando → va a la ECBLIST del bucle (L02)
3	En el COM: COMCIBPT	Dirección del primer CIB (mapeo IEZCIB): el comando en sí
4	Leer CIBVERB	Qué fue: START , MODIFY (F ZTKMON, texto) o STOP (P ZTKMON)
5	Leer CIBDATA / CIBDATLN	El texto del MODIFY, por ejemplo ESTADO
6	QEDIT ORIGIN=..., BLOCK=...	Liberar el CIB ya atendido (si no, se acumulan)
7	QEDIT ORIGIN=..., CIBCTR=n	Permitir hasta <i>n</i> MODIFY encolados

Comando	Qué haría ZTKMON (propuesta)
F ZTKMON, ESTADO	WTO con vueltas, eventos, clientes conectados y CPU
P ZTKMON	Cerrar el socket, mandar +FIN, cerrar SYSPRINT y terminar con RC 0

 **Analogía: el buzón de órdenes de la oficina**

Cuando el gerente deja una orden, se levanta la banderita (el ECB del COM). El empleado abre el buzón, lee el papel (el CIB), hace lo que dice y **tira el papel** (QEDIT); si no lo tira, el buzón se llena.

Los valores exactos de CIBVERB se toman del mapeo IEZCIB al escribir la lección.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Communicating with the system operator": EXTRACT, QEDIT, CIB)

Lámina L07-02_comandos_del_operador

4. El bucle final

Con esto, la **ECBLIST** de L02 queda completa:

ECB	Lo postea	Acción
Timer	La salida de STIMERM	Una vuelta de SSI 80
COM (COMECBPT)	El sistema, al llegar F / P	Leer el CIB, actuar y liberarlo
Socket (L06)	La API, si se usa el modo con ECB	Leer el pedido de la PC
Evento (L08)	El listen exit de ENF 70	Mandar +EVT

5. Recuperación: ESTAEX

Sin recuperación, unabend mata el STC sin dejar rastro en el monitor, y el portal se entera sólo por COLECTOR caído.

Con **ESTAEX**:

- El sistema llama a **tu rutina de recuperación**, con el diagnóstico en el **SDWA**.
- La rutina puede **reintentar** en un punto seguro (*retry*) o dejar seguir elabend (*percolate*).
- **Para un monitor, la política sugerida** es registrar (WTO + SYSPRINT), cerrar el socket ordenado y **terminar**. No hay que ocultar el error.

 Fuente: *Assembler Services Guide*, "Providing recovery" (ESTAEX, SDWA, retry y percolate).

Sin recuperación	Con <code>ESTAE</code>	Qué ganamos
El abend mata el STC; el socket queda abierto hasta que TCP/IP lo limpie	El sistema llama a tu rutina de recuperación con el diagnóstico (SDWA)	Escribir qué pasó (WTO + SYSPRINT)
El portal se entera por COLECTOR caído (180 s)	La rutina puede reintentar en un punto seguro (<i>retry</i>) o dejar que siga el abend (<i>percolate</i>)	Mandar <code>+ERR</code> o cerrar el socket ordenado
No hay rastro de la causa en el monitor	Para un monitor: registrar, cerrar limpio y terminar (no ocultar el error)	Que el operador vea un mensaje claro



Analogía: el paracaídas de reserva

ESTAE es el paracaídas de reserva: no evita que algo falle, pero permite **aterrizar ordenado** y dejar dicho qué pasó.

Reentrancia (repaso de L01): un STC de varias tareas debería tener sus datos de trabajo en memoria propia (`STORAGE OBTAIN`), no dentro de la CSECT.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Providing recovery": ESTAE, SDWA, retry y percolate)

Lámina L07-03_recuperacion_estae

6. Reentrancia y memoria propia

En L01 vimos que ZTKMONS v0 tiene sus datos **dentro de la CSECT**. Para el STC conviene:

- pedir el área de trabajo con `STORAGE OBTAIN` al arrancar (y `STORAGE RELEASE` al terminar);
- apoyar los datos con un `USING` sobre una DSECT;
- dejar la CSECT **sin modificar**.

Así, si en algún momento hay más de una tarea (por ejemplo, el exit de L08), nadie escribe sobre los datos de otro.

7. Preguntas de comprobación

1. ¿Qué pasa si nunca liberás los CIB con QEDIT y el operador manda veinte `F ZTKMON, ESTADO` ?
2. ¿Por qué el ECB del COM tiene que estar en la `ECBLIST` y no consultarse cada tanto?
3. ¿Qué diferencia hay entre *retry* y *percolate*? ¿Cuál conviene para un monitor y por qué?
4. ¿Qué tiene que pedirle a sistemas el STC que no pedía el job batch?

5. ¿Por qué una CSECT que se modifica a sí misma es un problema cuando hay más de una tarea?

Mis respuestas

8. Práctica P07: ZTKOIDO (sin pedir nada a sistemas)

Un **job batch** que:

1. Hace `EXTRACT ... FIELDS=COMM` y `QEDIT ... CIBCTR=5`.
2. Espera en una `ECBLIST` con el ECB del COM y un timer de 5 minutos.
3. Ante `F ZTK0ID0,ESTADO` contesta con un `WTO` (vueltas, hora de inicio).
4. Ante `P ZTK0ID0` termina ordenado, con RC 0.
5. Si pasan los 5 minutos sin orden, termina con RC 4.

A confirmar en DESA: que F y P lleguen también a un job batch que hizo EXTRACT ... COMM. Es lo esperable, pero lo prueba esta práctica.

Traé el JOBID, el SYSLOG o JESMSGLG con tus **F / P** y sus respuestas, y el RC final.

Relacionado: [L02 - Dormir sin gastar CPU - STIMER, WAIT, POST y ECB](#) · [L06 - Sockets en z-OS](#) · siguiente: [L08 - Eventos de verdad - ENF 70](#).



Fuente: *Assembler Services Guide* (EXTRACT, QEDIT, ESTAEX, STORAGE) · *z/OS MVS System Commands* (START, MODIFY, STOP) · *z/OS Security Server RACF* (clase STARTED).

L08 — Eventos de verdad: ENF 70

Objetivo: pasar del nivel B (preguntar cada N segundos) al nivel C, en el que **JES avisa** cuando un job arranca o termina. Al final sabés cómo se escucha la señal, qué reglas tiene el exit y por qué el polling no se tira.

 **Requiere de sistemas:** que el programa venga de una **librería APF** y corra como **STC** (L07).

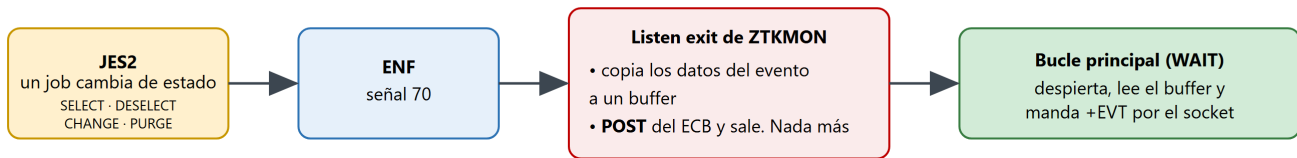
1. Qué es ENF

ENF (*Event Notification Facility*) es el mecanismo con el que z/OS avisa eventos a quien se haya anotado para escucharlos. Cada tipo de evento tiene un **código**. El **70** lo emite **JES2** cuando un job cambia de estado:

Evento	Qué significa para nosotros
SELECT	El job fue tomado para ejecutar → +EVT INI
DESELECT	Terminó de ejecutar, con el motivo (normal, CC,abend, JCL, cancelado, flush) → +EVT FIN
CHANGE	Cambió algún atributo o fase → +EVT FASE (opcional)
PURGE	Se purgó → +EVT PURGA (opcional)

Los datos del evento se leen con el mapeo **IAZENF70** de SYS1.MACLIB.

 Fuente: *z/OS MVS Programming: Authorized Assembler Services Reference* (ENFREQ) · mapeo IAZENF70 · IDEA-005, nivel C.



Al arrancar: ENFREQ ACTION=LISTEN, CODE=70, EXIT=... · al parar (P ZTKMON): ENFREQ ACTION=DELETE
 Los datos del evento se leen con el mapeo IAZENF70 (SYS1.MACLIB).

🔔 Analogía: el timbre del edificio (nivel C de L00)

En vez de que el vigilante haga rondas, JES **toca el timbre** cada vez que un job entra o sale. El exit es el **timbre**: sólo suena y deja una nota; no puede ponerse a conversar. El que atiende es el bucle principal.

Fuente: z/OS MVS Programming: Authorized Assembler Services Reference (ENFREQ) · mapeo IAZENF70 · IDEA-005, nivel C

Lámina L08-01_flujo_enf70

2. Anotarse y borrarse

Momento	Llamada	Qué hace
Al arrancar	ENFREQ ACTION=LISTEN, CODE=70, EXIT=...	Registra tu exit para la señal 70 y devuelve un token de la escucha
Al parar (P ZTKMON)	ENFREQ ACTION=DELETE con ese token	Deja de escuchar

Si no hacés el DELETE, la señal sigue apuntando a un exit que ya no existe.
 Por eso el DELETE va también en la rutina de recuperación (L07).

3. La autorización

ENFREQ ACTION=LISTEN es un servicio **autorizado**. El que lo llama tiene que estar en **estado supervisor** o en **clave PSW 0–7**.

- Un programa de una **librería APF**, enlazado con AC=1, puede pasar a estado supervisor con MODESET, hacer el pedido y **volver enseguida** a estado problema.
- **Principio:** estar autorizado sólo el tiempo mínimo. Cuanto menos código corre autorizado, menos daño puede hacer un error.

4. El listen exit: reglas de oro

Regla	Por qué
Corto: copiar los datos y hacer POST	Corre en el contexto del que emite la señal: si se demora, demora a JES
Sin I/O ni esperas	Nada de WTO largos, PUT ni sockets: eso lo hace el bucle principal
Pensado para ráfagas	En el pico del cierre llegan muchos eventos seguidos
Dónde tiene que residir el exit y en qué modo corre	A confirmar en el manual de ENFREQ antes de escribirlo (requisitos de memoria del exit)

5. El buffer entre el exit y el bucle

El exit y el bucle principal **corren a la vez**, así que comparten un buffer con cuidado:

- Un **buffer circular** de N entradas, con dos índices: **escribir** (lo mueve el exit) y **leer** (lo mueve el bucle).
- El exit avanza su índice con **CS** (*compare and swap*), una instrucción atómica. Así dos eventos simultáneos no pisan la misma entrada.
- **Si el buffer se llena**, el exit no espera: marca una bandera de "desborde" y descarta. Al ver la bandera, el bucle **reconcilia con SSI 80** (**ACT** + **POSTEX**) y nada se pierde de verdad.

Criterio propio. Es el diseño a validar en la práctica; el tamaño del buffer se elige con la ráfaga de L04 (50 jobs).

6. Nivel B vs. nivel C

	B · SSI 80 cada N s	C · ENF 70
Latencia	Hasta N segundos	Inmediata
CPU sin actividad	Una consulta por vuelta	≈ 0
Permisos	Ninguno	APF + STC
Si se pierde un evento	La próxima vuelta lo ve	Hay que reconciliar con SSI 80

Por eso B no se tira: en el nivel C, SSI 80 sigue sirviendo para reconciliar al arrancar, después de un desborde y en cada reconexión (ACT, HIST).

Las reglas del listen exit y de la autorización			L08 · Eventos de verdad: ENF 70
Regla		Por qué	
El exit es corto: copiar y POST		Corre en el contexto del que emite la señal: si se demora, demora a JES	
Sin I/O ni esperas en el exit		Nada de WTO largos, PUT ni sockets: eso lo hace el bucle	
El buffer entre exit y bucle debe tolerar ráfagas		En el pico del cierre pueden llegar muchos eventos seguidos; hay que decidir qué pasa si se llena	
ENFREQ LISTEN exige estar autorizado		Estado supervisor o clave 0-7: el programa tiene que venir de una librería APF	
Dónde debe residir el exit		A confirmar en el manual al llegar a L08 (requisitos de memoria del exit para ENFREQ)	
Al terminar: ENFREQ ACTION=DELETE		Si no, la señal sigue apuntando a un exit que ya no existe	
Comparación		Nivel B (SSI 80 cada N s)	Nivel C (ENF 70)
Latencia		Hasta N segundos	Inmediata
CPU sin actividad		Una consulta por vuelta	≈ 0
Permisos		Ninguno	APF + STC
Si se pierde un evento		La próxima vuelta lo ve	Hace falta reconciliar con SSI 80 (ACT / HIST)
Por eso B no se tira: en el nivel C, SSI 80 sigue sirviendo para reconciliar al arrancar y después de cualquier corte.			
Fuente: Authorized Assembler Services Reference (ENFREQ) · criterio propio para la comparación (se mide en la práctica)			

Lámina L08-02_reglas_del_exit

7. Preguntas de comprobación

1. ¿Por qué el exit no puede escribir directamente en el socket?
2. ¿Qué pasa si dos eventos llegan al mismo tiempo y el exit avanza el índice con L / A / ST en vez de CS ?
3. ¿Qué hacés si el buffer se llena en el pico del cierre?
4. ¿Por qué hay que hacer ENFREQ DELETE también en la rutina de recuperación?
5. Con el nivel C funcionando, ¿para qué seguís necesitando SSI 80?

L09 — ¿Quién tiene el archivo? ENQ y GQSCAN

Objetivo: entender los "candados" de z/OS y saber **mirar quién tiene un dataset sin tocarlo**. Con eso, el monitor puede avisar **antes** de que falle un job como FDCICS02.

1. ENQ: los candados de z/OS

Concepto	Qué es	Ejemplo
Recurso	Un nombre en dos partes: QNAME (8 bytes) + RNAME (hasta 255)	SYSDSN + nombre del dataset
Exclusivo / compartido	E: uno solo. S: varios a la vez, pero ninguno exclusivo	DISP=OLD → E · DISP=SHR → S
Alcance	STEP · SYSTEM · SYSTEMS (todo el sysplex, vía GRS)	Las asignaciones de datasets se serializan con SYSDSN
Dueño / en espera	Quien lo tiene vs. quien hace cola	—

Cuando un job **asigna** un dataset, el sistema pide el ENQ `SYSDSN` con el nombre del dataset: compartido si es `SHR`, exclusivo si es `OLD`, `MOD` o `NEW`. Si otro lo tiene de forma incompatible, la asignación espera o falla, según cómo se pidió.

 Fuente: z/OS MVS Programming: Assembler Services Guide, "Serializing access to resources" (ENQ, DEQ, GQSCAN).

Concepto	Qué es	Ejemplo
Recurso	Un nombre en dos partes: QNAME (8 bytes) + RNAME (hasta 255)	SYSDSN + nombre del dataset
Exclusivo / compartido	E = uno solo; S = varios a la vez (pero ninguno exclusivo)	DISP=OLD → E · DISP=SHR → S
Alcance	STEP · SYSTEM · SYSTEMS (todo el sysplex, vía GRS)	SYSDSN se propaga como SYSTEMS
Dueño / esperando	Quien lo tiene vs. quien hace cola	CICSFILE dueño; FDCICS02 quiere E

Caso real del 05-10 (FDCICS02, JOB29789): el DELETE pidió el dataset en exclusivo y la asignación respondió IN USE BY ... CICSFILE ASID 0079 (IEFA110I DATA SET CONTENTION). Eso es contención por el recurso SYSDSN .

Analogía: los casilleros con candado

Cada dataset tiene un casillero. **Compartido** es un candado con varias llaves: muchos pueden mirar. **Exclusivo** es una sola llave: el DELETE necesita la única llave, y mientras CICS tenga una llave puesta, no la consigue.

Fuente: z/OS MVS Programming: Assembler Services Guide ("Serializing access to resources": ENQ, GQSCAN) · spool de JOB29789 (05-10)

Lámina L09-01_enq_sysdsn

2. El caso real: FDCICS02, 05-10 (JOB29789)

Hora	Qué pasó
00:00:02	PASO1 manda CEMT S FILE(FD*) CLO a CICSFILE y a CICSPEP, con RC 00
00:00:03	FDMCTA: DELETE FOP.DOLARES.TR.M.P.MASTER.CUENTAS → IKJ56225I ... ALREADY IN USE · IN USE BY ... CICSFILE ASID 0079 · IEFA110I DATA SET CONTENTION
00:00:03	El DELETE no se hace. Después, DEFINE → IDC3013I DUPLICATE , RC 12
00:01	El rerun pasa: CICS ya soltó el archivo

Lectura con ENQ: el DELETE necesitaba el dataset en **exclusivo**, y CICSFILE todavía lo tenía asignado. El CEMT CLOSE es asíncrono: el paso que lo manda termina antes de que la región suelte el dataset.

Historia medida: 6 fallas en 100 noches, todas a las 00:00–00:01, y 4 de 15 domingos contra 2 de 85 en otros días (Fisher $p \approx 0,004$, **exploratorio**).

3. GQSCAN: mirar los candados sin tocarlos

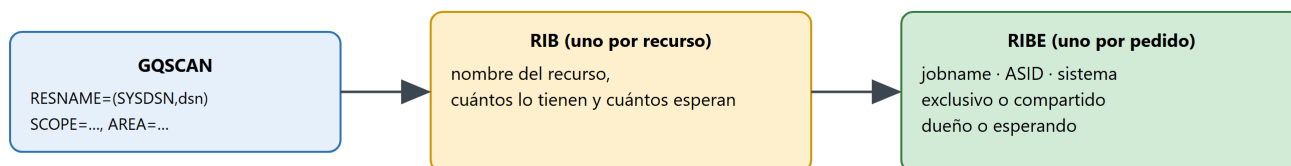
GQSCAN AREA=(área,largo),SCOPE=...,RESNAME=(SYSDSN,nombre-del-dataset,largo)

Respuesta	Qué trae
RIB (uno por recurso)	El nombre del recurso, cuántos lo tienen y cuántos esperan
RIBE (uno por pedido)	jobname, ASID, sistema, exclusivo o compartido, dueño o en espera

- Mapeo de la respuesta: **ISGRIB**.
- Si el área no alcanza, GQSCAN devuelve un **token** para seguir leyendo. El código de retorno exacto de cada caso se toma del manual al escribir la práctica.

GQSCAN: mirar los candados sin tocarlos

L09 · ¿Quién tiene el archivo? ENQ y GQSCAN



Mapeo de la respuesta: ISGRIB. Si el área no alcanza, GQSCAN devuelve un token para seguir leyendo.

Regla de oro	Por qué
GQSCAN sólo mira	No pide ningún candado: no puede trabar a nadie
Nunca hacer ENQ sobre un recurso de producción "para ver"	El monitor se metería en la cola y podría demorar a un job real
¿Alcanza el estado problema (sin APF)?	No probado. Según el manual estaría disponible sin autorización: se prueba en la práctica



Analogía: mirar el tablero de llaves de la portería

GQSCAN es mirar el tablero de llaves para saber quién tiene cada una, **sin tocar ninguna llave**.

Fuente: Assembler Services Guide (GQSCAN, mapeo ISGRIB) · IDEA-005, complemento "mutex"

Lámina L09-02_gqscan

4. Reglas de oro

Regla	Por qué
GQSCAN sólo mira	No pide ningún candado: no puede trabar a nadie
Nunca hacer ENQ sobre un recurso de producción "para ver"	El monitor se metería en la cola y podría demorar a un job real
Probar sólo con datasets proprios de prueba	Nunca mirar ni tocar datos de producción para practicar

¿Alcanza el estado problema (sin APF)?

No probado. Según el manual estaría disponible sin autorización; lo confirma la práctica

5. La alerta de contención (IDEA-008) en vivo

1. Arranca un job que, según la historia, falla por archivo ocupado: FDCICS02, NCPD1K95, CIFT00L2...
2. El monitor pregunta con GQSCAN **quién tiene ahora** los datasets que ese job va a borrar o abrir en exclusivo (sale del mapa de archivos, `mapa.db`).
3. Si el dueño es una región CICS, el portal avisa **antes** de la falla, con el dueño y el ASID.

Lo que ya sabemos: la regla genérica "IDCAMS después de un CEMT" **no es significativa** ($p \approx 0,12$, 101 noches). Por eso conviene una **lista de jobs con historia**, no una regla de forma.

El monitor **avisa**; no espera ni reintenta por el job. Agregar una espera después del CEMT es una decisión de desarrollo.

La alerta de contención (IDEA-008) en vivo

L09 · ¿Quién tiene el archivo? ENQ y GQSCAN

#	Qué detecta el monitor	Con qué
1	Arranca un job que, según la historia, falla por archivo ocupado (FDCICS02, NCPD1K95, CIFT00L2...)	+EVT INI + lista de jobs con historia (portal)
2	¿Quién tiene ahora mismo los datasets que ese job va a borrar o abrir en exclusivo?	GQSCAN sobre <code>SYSDSN</code> + el mapa de archivos (<code>mapa.db</code>)
3	Si el dueño es una región CICS, avisar antes de que falle	Alerta "CONTENCIÓN" en el portal, con el dueño y el ASID

Lo que ya sabemos (medido)	Valor
FDCICS02: fallas por FDMCTA en uso por CICSFILE	6 en 100 noches, todas a las 00:00–00:01; 4 de 15 domingos contra 2 de 85 en otros días (Fisher $p \approx 0,004$, exploratorio)
Regla genérica "IDCAMS después de un CEMT"	No significativa ($p \approx 0,12$): conviene una lista de jobs con historia

El monitor **avisa**; no espera ni reintenta por el job. Cambiar el JCL (por ejemplo, una espera después del CEMT) es decisión de desarrollo.

Fuente: Informe de cierres 02–04–10 (sección 3) · IDEAS_cierres_produccion.md, IDEA-008

Lámina L09-03_alerta_contencion

6. Preguntas de comprobación

1. ¿Por qué un `DELETE` necesita el dataset en exclusivo y una lectura no?

- ## Mis respuestas

1. **ZTKTOMA:** un job que asigna un dataset **tuyo** de prueba con `DISP=OLD` y duerme 5 minutos (tu ZTKBUCLE de P02 sirve).
2. **ZTKQUIEN:** mientras ZTKTOMA duerme, un programa que hace GQSCAN sobre `SYSDSN` + ese dataset y escribe por cada RIBE el jobname, el ASID, si es exclusivo o compartido y si es dueño o espera.

Traé el SYSPRINT de ZTKQUIEN. Tiene que mostrar a ZTKTOMA como **dueño exclusivo**. También contestá la pregunta abierta: ¿anduvo **sin APF**?

Relacionado: [L08 - Eventos de verdad - ENF 70](#) · siguiente: [L10 - El lado PC - puente, portal y ML en línea](#).

 Fuente: *Assembler Services Guide* (ENQ, GQSCAN, mapeo ISGRIB) · spool de JOB29789 (cierres\2026-10\2026-10-04\spool\) · informe de cierres del 02 al 04-10 · IDEA-008.

L10 — El lado PC: puente, portal y ML en línea

Objetivo: entender qué pasa con cada `+EVT` desde que sale del host hasta que se convierte (o no) en una alerta, y **cómo aprende el portal** cuánto dura normalmente cada job. Esta parte ya existe y funciona contra el simulador; la meta del módulo es probarla contra el monitor real.

1. Del evento a la alerta



- `vivo.db` es aparte de `cierres.db`. La ingesta nocturna escribe en una y el monitor en la otra, sin pisarse.
- **Cada entorno aprende por separado:** DESA no se mezcla con PROD.

Del evento a la alerta

L10 · El lado PC: puente, portal y ML en línea

```
graph LR; A["+EVT FIN ... (EBCDIC)"] --> B["puente.py<br/>traduce a JSON"]; B --> C["POST /api/vivo/observaciones<br/>vivo.py -> datos\vivo.db"]; C --> D["Evaluador (cada 10 s)<br/>COLGADO · FALLA · COLECTOR"];
```

Cada alerta abre un estudio (historia del job, puesto en el ranking de riesgo, sucesores en TWS, mensajes clave del spool).

Alerta	Cuándo salta
COLGADO	El job lleva más que su umbral aprendido
FALLA	Terminó con CC ≥ 8,abend o error de JCL
COLECTOR caído	180 s sin datos ni latido

Fuente: Impl - Monitoreo en vivo (cerebro ml-cierres) · app\vivo.py

Lámina L10-01_del_evento_a_la_alerta

2. El puente (`vivo\puente.py`)

Regla	Valor
-------	-------

Código de caracteres	Decodifica con <code>cp037</code> ; cada línea termina en <code>X'15'</code>
Al conectar	<code>SUB</code> y <code>ACT</code> , más <code>HIST</code> con la última hora que guarda el portal
Latido	<code>PING</code> cada 30 s
Reconexión	Sin <code>+PONG</code> en 90 s, reconecta con esperas de 5, 10, 20, 40 y 60 s
Salida	Cada línea → <code>JSON</code> → <code>POST</code> al portal

3. El portal (`app\vivo.py`)

Alerta	Cuándo
COLGADO	El job lleva corriendo más que su umbral
FALLA	Terminó con $CC \geq 8$, abend o error de JCL (cancelado = severidad MEDIA)
COLECTOR	180 s sin latido ni eventos

- Hay **una sola alerta abierta** por entorno, jobid y tipo.
- El **evaluador** corre cada 10 s.
- Todos los parámetros están en `CFG` y se pueden cambiar con variables `VIVO_<CLAVE>` (por ejemplo, para pruebas).

4. Cómo aprende el umbral de colgado

Situación	Umbral
Con ≥ 5 corridas del job (<code>MIN_HIST</code>)	Kaplan–Meier en segundos: el primer x con $P(T \geq x - 5 \text{ s}) < 1\%$ (<code>ALFA</code> 0,01, <code>TOL_S</code> 5). Los abends y cancelaciones cuentan como censura . Además, un piso: máx(KM, 2 × mediana, mediana + 60 s)
Arranque en frío	máx(300 s, 5 × mediana del entorno) (<code>PISO_FRIIO_S</code> , <code>FACTOR_FRIIO</code>)
PROD	Si el job tiene umbral en <code>m1_umbral</code> (aprendido de 90 cierres), se usa ése
Recalcular	En cada vuelta: si aprende algo mientras el job corre, lo usa

Kaplan–Meier en dos líneas: estima la probabilidad de que el job "siga vivo" después de x segundos, usando también las corridas que no terminaron normalmente (la censura) sin tratarlas como duraciones reales. El umbral es el punto en que esa probabilidad baja del 1 %.

Cómo aprende el umbral de colgado

L10 · El lado PC: puente, portal y ML en línea

Situación	Umbral
Con ≥ 5 corridas del job	Kaplan–Meier en segundos: el primer x con $P(T \geq x - 5 \text{ s}) < 1 \%$ (los abends cuentan como censura), con un piso: máx(2 × mediana, mediana + 60 s)
Arranque en frío (job nuevo)	máx(300 s, 5 × mediana del entorno)
PROD	Usa <code>m1_umbral</code> (90 cierres)
Entornos	DESA y PROD no se mezclan
Recalcular	En cada vuelta: si aprende algo mientras el job corre, lo usa

Lo que salió mal y se corrigió: la corrida colgada entró a la base y subió el umbral de ZTKDOK de **66 a 155 s**: el próximo colgado no se habría visto. Ahora la corrida que dispara COLGADO queda marcada como anómala y **no se aprende de ella** (el botón «Era normal» la devuelve).

 **Analogía: el panadero y el horno**

El panadero aprende cuánto tarda cada pan. Si un día el horno se rompe y el pan tarda el triple, **no anota ese día** como "lo normal": si lo hiciera, la próxima vez que se rompa el horno no se daría cuenta.

Fuente: Impl - Monitoreo en vivo y Impl - Supervivencia (cerebro ml-cierres) · prueba_e2e.py 11/11 (2026-09-30)

Lámina L10-02_umbral_de_colgado

5. Lo que salió mal y cómo se arregló

Problema (medido en la prueba del 30-09)	Arreglo
La corrida colgada entró a la base y subió el umbral de ZTKDOK de 66 a 155 s : el próximo colgado no se habría visto	La corrida que dispara COLGADO queda marcada <code>anomala=1</code> y no se aprende de ella ; el botón «Era normal» la devuelve
Un FIN sin INI quedaba con duración 0	El FIN trae <code>TI=</code> ; sin él, no entra al aprendizaje
Reconexión: 55 de 63 eventos recuperados	HIST desde la hora que guarda el portal: 63 de 63
COLECTOR en falso	El plazo (180 s) tiene que ser mayor que el latido (30 s)

6. Límites honestos

Límite	Qué lo compensaría
No hay CPU por job: "lleva xN su umbral" no es lo mismo que "está en loop"	El estudio suma el R-LOOP del código (mapa de programas, IDEA-006)
No hay paso actual: ENF 70 y SSI 80 son por job	Otra fuente, a investigar
ATRASO contra el plan TWS no está implementado, porque DESA no tiene plan	Implementarlo en PROD con <code>plan_op</code>
La regresión corre contra el simulador	Repetirla contra el monitor real: es la meta de este módulo

Límites honestos del lado PC

L10 · El lado PC: puente, portal y ML en línea

Límite	Por qué	Qué lo compensaría
Sin CPU por job	"Lleva xN su umbral" no es lo mismo que "está en loop"	El estudio suma el R-LOOP del código (mapa de programas, IDEA-006)
Sin paso actual	ENF 70 y SSI 80 son por job, no por paso	Otra fuente (a investigar)
ATRASO contra el plan TWS	DESA no tiene plan	Implementarlo en PROD con <code>plan_op</code>
Regresión	<code>prueba_e2e.py</code> corre contra el simulador	Repetirla contra el monitor real (meta del módulo 10)

Fuente: Impl - Monitoreo en vivo (cerebro ml-cierres), sección "Límites"

Lámina L10-03_limites_del_lado_pc

7. Preguntas de comprobación

1. ¿Por qué el monitor no debe aprender de la corrida que disparó COLGADO?
2. ¿Qué es la "censura" en Kaplan–Meier y por qué unabend se trata así?
3. Un job nuevo con mediana de entorno 40 s: ¿cuál es su umbral de arranque en frío?
4. ¿Por qué `vivo.db` está separada de `cierres.db`?
5. ¿Qué cambia en el umbral si el job es de PROD y tiene fila en `ml_umbral`?

L11 — Operación y pase

Objetivo: dejar el monitor listo para que otros lo operen. Implica saber qué hay que pedir, qué medir antes, qué hacer cuando algo falla y cómo se propone el paso de DESA a PROD **sin que deje de ser de sólo lectura**.

1. Qué hay que pedirle a sistemas, y cuándo

Pedido	Para qué	Desde
Puerto TCP reservado	Que ZTKMON escuche	L06
Regla de firewall PC ↔ host	Que la conexión llegue	L06
PROC en la PROCLIB	Poder hacer S ZTKMON	L07
Perfil en la clase STARTED	El usuario con el que corre	L07
Librería APF	ENFREQ LISTEN (ENF 70)	L08
Gestión de cambios	Pasar de DESA a PROD	L11

Se piden de a uno y recién cuando hacen falta, cada uno con la evidencia de la práctica anterior. Pedir todo junto al principio es pedir sin poder mostrar para qué.

Qué hay que pedirle a sistemas, y para qué

L11 · Operación y pase

Pedido	Para qué	Desde qué módulo
Puerto TCP reservado	Que ZTKMON escuche y la PC se conecte	6
Regla de firewall PC ↔ host	Que la conexión llegue	6
PROC en la PROCLIB	Poder hacer S ZTKMON	7
Perfil en la clase STARTED	El usuario con el que corre el STC	7
Librería APF	Poder usar ENFREQ LISTEN (ENF 70)	8
Gestión de cambios	Pasar de DESA a PROD	11

🔑 Analogía: las llaves de un local nuevo

Para abrir un local hace falta la llave de la puerta (puerto), el permiso municipal (STARTED), el registro del negocio (PROCLIB) y, para ciertas actividades, una licencia especial (APF). Se piden de a una, recién cuando hacen falta.

Fuente: Ruta de aprendizaje (columna "Requiere de sistemas")

Lámina L11-01_que_pedir_a_sistemas

2. Qué se mide antes del pase

Medida	Cómo	Contra qué
CPU por vuelta (nivel B)	CPU del STC ÷ vueltas (L02, L04)	El costo de no hacer nada (P02)
CPU por evento (nivel C)	CPU del STC ÷ eventos (L08)	La CPU del nivel B
Latencia del aviso	Hora del +EVT vs. IEF403I / \$HASP395	El intervalo de B
Eventos perdidos	spool_a_portal.py y prueba_e2e.py contra el monitor real	Cero
Ráfaga	50 jobs seguidos (L04)	Cero perdidos

3. Runbook: qué hacer si...

Si...	Qué se ve	Qué hacer
El monitor se cae	Alerta COLECTOR caído (180 s)	S ZTKMON ; al reconectar, el puente pide ACT + HIST y no se pierde nada
Hay que pararlo	—	P ZTKMON . Nunca cancelar si no hace falta
Querés saber cómo anda	—	F ZTKMON, ESTADO
El puente de la PC se cae	El host sigue; el portal marca COLECTOR	Relanzar el puente; recupera con HIST
Gasta demasiada CPU	CPU del STC en el JESMSG LG o en RMF	Subir el intervalo (nivel B) o pasar al nivel C
Abend del monitor	Mensaje de la rutina ESTAEX en consola y SYSPRINT	Juntar el mensaje y el dump, S ZTKMON , y analizar

Si...	Qué se ve	Qué hacer
El monitor se cae	Alerta COLECTOR caído en el portal (180 s)	S ZTKMON ; al reconectar, el puente pide ACT + HIST y no se pierde nada
Hay que pararlo	—	P ZTKMON (nunca cancelar si no hace falta)
Querés saber cómo anda	—	F ZTKMON, ESTADO
El puente de la PC se cae	El host sigue; el portal marca COLECTOR	Relanzar el puente; recupera con HIST
El monitor gasta demasiada CPU	CPU del STC en RMF o en el JESMSG LG	Subir el intervalo (nivel B) o pasar al nivel C

Se mide antes del pase	Cómo
CPU por vuelta y por evento	CPU del paso ÷ vueltas (L02, L04)
Latencia del aviso	Hora del +EVT vs. IEF403I / \$HASP395
Eventos perdidos	spool_a_portal.py y prueba_e2e.py contra el monitor real

Principio que no cambia en PROD: el monitor **sólo mira**. Ninguna versión cancela, libera ni toca TWS.

Fuente: PROTOCOLO.md · Ruta de aprendizaje (módulo 11) · criterio propio para el runbook

Lámina L11-02_runbook

4. De DESA a PROD

1. **Demo en DESA** con las cinco preguntas contestadas (L04) y la regresión 11/11 contra el monitor real (L10).
2. **Informe** con las mediciones de la sección 2, lo que salió mal y cómo se arregló.
3. **Propuesta de cambio:** PROC, STARTED, puerto, firewall y APF si va el nivel C, más el runbook.
4. **Piloto en PROD** con el filtro más chico posible (unos pocos jobs) y el portal en modo "sólo mirar".
5. **Ampliar** el filtro de a poco, midiendo CPU y latencia en cada paso.

Principio que no cambia en PROD: el monitor **sólo mira**. Ninguna versión cancela, libera ni toca TWS. Eso no se negocia en el pase.

5. Riesgos y cómo se mitigan

Riesgo	Mitigación
Que el monitor demore a JES (nivel C)	Exit corto, sin I/O; buffer con desborde que se reconcilia con SSI 80 (L08)

Que el monitor trabee un dataset	GQSCAN sólo mira; nunca ENQ sobre recursos de producción (L09)
Que consuma CPU en el pico del cierre	Medido antes del pase; intervalo ajustable; nivel C
Que se conecte quien no debe	Lista de IPs en el host + firewall
Que se pierdan eventos en un corte	ACT + HIST al reconectar (L05)

6. Preguntas de comprobación

1. ¿Por qué conviene pedir los permisos de a uno y no todos juntos?
2. ¿Qué tres números presentarías para justificar el pase a PROD?
3. Si el monitor gasta el doble de CPU de lo esperado en el pico del cierre, ¿qué dos palancas tenés?
4. ¿Por qué el piloto en PROD empieza con un filtro chico?
5. ¿Qué parte del diseño garantiza que el monitor no puede afectar un job de producción?

